COMPUTATIONAL PATHS: A CALCULUS OF EQUALITY

A COMPREHENSIVE SURVEY

ARTHUR FREITAS RAMOS, RUY J.G.B. DE QUEIROZ,
AND ANJOLINA GRISI DE OLIVEIRA

Abstract. We survey the theory of *computational paths*—a proof-relevant account of equality in type theory in which every equality witness carries an explicit trace of elementary rewrite steps. Starting from the framework of labelled natural deduction, where the judgement $t =_s u : A$ records *why* two proof-terms are equal, we develop a complete rewrite calculus on the resulting path expressions. The calculus comprises 77 named rewrite rules, organised into eight categories covering the groupoid core, type constructor interactions, transport, and congruence. The system is proved to be both terminating and confluent, yielding unique normal forms and decidable path equivalence.

The higher algebraic structure of computational paths is then developed in stages: paths between terms form a weak groupoid; rewrite derivations between paths (2-cells) give rise to a weak 2-groupoid with vertical and horizontal composition; 3-cells provide coherence data including the pentagon and triangle identities; and the entire tower assembles into a weak ω-groupoid in the sense of Batanin–Leinster, with stabilisation above dimension 3. The Eckmann–Hilton argument establishes commutativity in the double loop space. We derive the J-eliminator (path induction) from the contractibility of the based path space, demonstrate the failure of the Uniqueness of Identity Proofs for the Path type, and compute fundamental groups of combinatorial spaces ($\pi_1(S^1) \cong \mathbb{Z}$, $\pi_1(T^2) \cong \mathbb{Z} \times \mathbb{Z}$, $\pi_1(\mathbb{R}P^2) \cong \mathbb{Z}/2\mathbb{Z}$). The central results have been formalised in Lean 4 (over 10,000 theorems, no axioms).

The full development, proofs, and formalisation details are presented in the body of this paper.

Date: March 2026.

2020 *Mathematics Subject Classification.* 03B15, 03B38, 03B70, 18N50, 55Q05, 68V15.

Key words and phrases. computational paths, weak ω-groupoid, labelled natural deduction, term rewriting, proof relevance, homotopy type theory, Lean 4.

1. Introduction

1.1. **The problem: equality and its reasons.** The Curry–Howard correspondence [11, 38] establishes a dictionary between logic and type theory: propositions correspond to types, proofs to programs, and proof simplification to computation (β-reduction). This dictionary has been enormously fruitful—it underlies modern proof assistants such as Coq [62], Agda [47], and Lean [12]—but it leaves a fundamental question unanswered. If proofs correspond to programs and proof simplification corresponds to computation, then what corresponds to *equality of computations*?

Consider two computations of the same result. Both begin with the same expression and end at the same normal form, but the intermediate steps may differ: one reduces the outermost redex first, the other an inner one. The Church–Rosser theorem [9] guarantees that the two reduction sequences can be brought to a common result, but says nothing about the sequences themselves. Are these witnesses to equality—these traces of computation—mathematically meaningful in their own right?

The theory of *computational paths* answers this question affirmatively. The sequences of rewrite steps that transform one term into another are reified as formal objects—computational paths—which are themselves subject to a rich algebraic theory. By extending the Curry–Howard dictionary one level further, we obtain:

Logic (extended)		**Type Theory (extended)**
Equality of proofs	$\leftrightarrow$	Path between terms
Reason for equality	$\leftrightarrow$	Computational path
Equality of reasons	$\leftrightarrow$	Path between paths (2-cell)

It is at the third row that the theory begins. The insight that "reasons for equality" can themselves be compared opens the door to an infinite hierarchy of higher-dimensional structure—precisely the structure of a weak ω-groupoid [34, 3, 42].

This hierarchy is intimately connected to a fundamental observation of Voevodsky [66]: the notion of equality itself must be stratified. Equality is appropriate for *elements*—individuals—but fails for collections; isomorphism is appropriate for collections but fails for collections of collections. This leads to a theory of iterated n-equivalences which are the correct replacements at each level. The ω-groupoid structure

that emerges from computational paths provides a concrete, syntactic realisation of precisely this hierarchy: 1-cells capture equality of terms, 2-cells capture equivalence of proofs, 3-cells capture coherence of equivalences, and so on through all finite levels.

1.2. **Historical roots and motivation.** The roots of this work lie in de Queiroz's introduction of the judgement form $t =_s u : A$ in the late 1980s [15, 16, 17], where the subscript s records the *rewrite reason*—an explicit, structured witness to the equality of proof-terms t and u. This innovation took place within the broader programme of labelled natural deduction (LND) [32, 31], which enriches traditional natural deduction by carrying proof-terms alongside formulas. The key observation was that rewrite reasons are not mere notational decorations but first-class mathematical entities: they can be composed, inverted, and compared.

Building on this foundation, de Queiroz and de Oliveira [22, 23, 13, 14] developed the equational fragment of labelled deduction over several decades, establishing normalisation procedures and proof transformations. The identification of the identity type with the type of computational paths was made explicit in [25, 52], and the full elaboration—including the 77-rule rewrite system, confluence, the ω-groupoid structure, and the Lean 4 formalisation—was carried out in [49, 26, 56, 54].

The dialogical reading of proof-term reduction, developed in [18, 20, 21], connects this programme to Wittgenstein's "meaning as use" principle: introduction rules play the role of a Defender's assertion, elimination rules play the role of an Attacker's challenge, and the reductum gives the continuation. This makes explicit the "meaning as use" principle: the meaning of each connective is fixed by the admissible challenges and by how those challenges are answered. The 77-rule system for computational paths can be seen as a complete specification of the "grammar of equality"—it tells us everything there is to know about how equality proofs can be combined, decomposed, and compared.

Remark 1.1 (Historical note). The idea that proof-term reduction carries semantically significant content goes back at least to Prawitz's *Natural Deduction* [48]. The further idea that the *paths* of reduction are worth studying as objects was advanced by de Queiroz [16, 17], building on earlier insights by Lambek and Scott [40], who wrote: "In writing $f : A \to B$ we think of f as the 'reason' why A entails B." The present work is the full elaboration of the programme implicit in this remark.

1.3. **Scope and organisation.** This paper presents a comprehensive survey of the theory of computational paths. It gives the main definitions, theorems, and constructions in a self-contained form suitable for a broad audience in mathematical logic, type theory, and related areas.

The paper is organised as follows. Section 2 presents the labelled natural deduction framework. Section 3 introduces computational paths and their fundamental operations. Section 4 describes the 77-rule rewrite system in full detail, presenting the complete rules for each category. Section 5 establishes confluence and termination with detailed proofs. Section 6 develops the groupoid structure with full definitions, proofs, and examples. Section 7 constructs the higher cells, the weak 2-groupoid structure, whiskering and horizontal composition, and the coherence conditions including the pentagon and triangle identities. Section 8 proves the weak ω-groupoid theorem with the stabilisation argument and the Eckmann–Hilton result. Section 9 derives path induction, analyses transport and dependent application, and establishes the failure of UIP. Section 10 presents applications to fundamental group computations, covering spaces, the Seifert–van Kampen theorem, and connections to homotopy type theory and cubical type theory. Section 11 reports on the Lean 4 formalisation with architectural details. Section 12 discusses open problems and future directions. Appendix A provides the complete list of all 77 rewrite rules in tabular form.

2. Labelled Natural Deduction

The framework within which computational paths arise is that of *labelled natural deduction* (LND), developed by de Queiroz and Gabbay [32] and subsequently extended by de Queiroz and de Oliveira [22, 23] over several decades. LND enriches ordinary natural deduction with an extra dimension: every formula in a deduction carries a *label*—a term from an associated functional calculus that records the structure of the proof.

2.1. **Three judgement forms.** While the standard Curry–Howard correspondence employs two basic judgement forms—$\Gamma \vdash t : A$ ("t proves A") and $\Gamma \vdash t = u : A$ ("t and u are equal proofs")—the LND framework introduces a third, richer form:

$$\Gamma \vdash t =_s u : A \tag{1}$$

where the subscript s is the *rewrite reason*: an explicit, structured witness to the equality of t and u.

Definition 2.1 (Labelled equality judgement)**.** A *labelled equality judgement* is an expression of the form $\Gamma \vdash t =_s u : A$, where Γ is a context,

t and u are proof-terms of type A, and s is a rewrite reason—a term in the language of paths denoting a specific sequence of rewrite steps transforming t into u.

The architecture of LND can be summarised as a two-dimensional system: the *horizontal* dimension consists of formulas and the logical calculus, while the *vertical* dimension consists of labels and the functional calculus. This parallels Frege's *Grundgesetze* [30] (terms) alongside *Begriffsschrift* [29] (formulas)—a point emphasised in [19].

2.2. **Proof-terms and reduction rules.** The base system is the simply-typed λ-calculus with products ($A \times B$), coproducts ($A + B$), the empty type ($\mathbf{0}$), and the unit type ($\mathbf{1}$). The β-reduction rules capture the computational content of proofs:

$$(\beta_{\to}) \qquad (\lambda x.\, t)\, u \to_\beta t[u/x]$$

$$(\beta_{\times,i}) \qquad \pi_i(\langle t_1, t_2 \rangle) \to_\beta t_i$$

$$(\beta_{+,1}) \qquad \mathsf{case}(\mathsf{inl}(t), x.u_1, y.u_2) \to_\beta u_1[t/x]$$

$$(\beta_{+,2}) \qquad \mathsf{case}(\mathsf{inr}(t), x.u_1, y.u_2) \to_\beta u_2[t/y]$$

Each β-rule expresses the harmony between introduction and elimination rules in the sense of Prawitz [48]: when an introduction is immediately followed by an elimination of the same connective, the resulting "detour" can be simplified.

The fundamental metatheoretic properties of the base system are:

Theorem 2.2 (Subject reduction)**.** *If $\Gamma \vdash t : A$ and $t \to_\beta u$, then $\Gamma \vdash u : A$.*

Theorem 2.3 (Church–Rosser for the base system)**.** *The β-reduction relation is confluent: if $t \to_\beta^* u_1$ and $t \to_\beta^* u_2$, then there exists v such that $u_1 \to_\beta^* v$ and $u_2 \to_\beta^* v$.*

The proof uses the parallel reduction technique of Tait and Martin-Löf [61]: one defines a "complete development" t^* that simultaneously reduces all redexes, and shows that any single parallel reduction step from t can be completed to t^*.

Remark 2.4 (Dialogical reading). The β-reduction rules admit a dialogical reading [18, 20, 21]: introduction rules play the role of a Defender's assertion, elimination rules play the role of an Attacker's challenge, and the reductum gives the continuation. This makes explicit the "meaning as use" principle: the meaning of each connective is fixed by the admissible challenges and by how those challenges are answered.

2.3. **Abstraction discipline and substructural logics.** A significant observation emphasised in Gabbay's LDS framework [31] is that the λ-abstraction rule admits variation. By restricting which abstractions are permitted, one obtains substructural logics:

Restriction on λ-abstraction	**Resulting logic**
Exactly one use (no vacuous, no contraction)	Linear logic [33]
At least one use (no vacuous abstraction)	Relevant logic
At most one use (no contraction)	Affine logic
No restriction	Intuitionistic logic

This shows that the computational interpretation is not limited to intuitionistic logic. The common thread is that *modus ponens* (application) remains unchanged while the introduction rules differ in their assumptions about resource use. The use of abstractors to make arbitrary names lose their identity goes back to Frege's *Grundgesetze* [30]: the binding transforms a name into a mere place-marker for arguments in substitutions [19].

2.4. **From reduction sequences to computational paths.** The transition from proof theory to the theory of computational paths is driven by a single observation: the reduction sequences of LND carry natural algebraic structure. Given sequences $p : t \rightsquigarrow u$ and $q : u \rightsquigarrow v$, one can compose them to form $p \cdot q : t \rightsquigarrow v$; given $p : t \rightsquigarrow u$, one can reverse it to form $p^{-1} : u \rightsquigarrow t$; and there is a trivial zero-step sequence $\mathsf{refl}_t : t \rightsquigarrow t$. These operations satisfy the laws of a *groupoid*—associativity, identity, and inversion—and the congruence operation $f_*(p)$ satisfies functoriality. Making this structure precise is the task of the next section.

Example 2.5 (Two distinct rewrite reasons)**.** Consider the term $t \equiv (\lambda x.\, (\lambda y.\, y\, x)\, z)\, w$. There are two natural reduction paths to the normal form $z\, w$:

Reason s_1 (outer-first):

$$(\lambda x.\, (\lambda y.\, y\, x)\, z)\, w \xrightarrow{\beta} (\lambda y.\, y\, w)\, z \xrightarrow{\beta} z\, w$$

Reason s_2 (inner-first):

$$(\lambda x.\, (\lambda y.\, y\, x)\, z)\, w \xrightarrow{\beta} (\lambda x.\, z\, x)\, w \xrightarrow{\beta} z\, w$$

Both reasons witness the same equality $t = z\, w$, but they differ in intermediate terms and reduction order. The theory of computational paths takes this difference seriously: s_1 and s_2 are genuinely different mathematical objects.

3. Computational Paths: Definition and Operations

3.1. **Steps, paths, and path expressions.** The construction proceeds in two stages. First, we identify the *atomic steps*—primitive rewrite operations. Second, we define *path expressions*—compound terms built from atomic steps using algebraic operations.

Definition 3.1 (Elementary rewrite step)**.** An *elementary rewrite step* in a type A is a triple (a, b, h) consisting of a source $a : A$, a target $b : A$, and a justification h recording one designated atomic equality step. We write $\mathsf{Step}(A)$ for the type of all steps in A.

The atomic steps for the simply-typed λ-calculus fall into two families:

- *Computational steps*: β-reduction, η-expansion, and their counterparts for products and sums.
- *Structural steps*: congruence rules (ξ-steps, μ-steps, ν-steps) that propagate computation through compound terms.

Definition 3.2 (Computational path)**.** A *computational path* from a to b in a type A is a pair $\langle \sigma, \pi \rangle$ where σ is a finite list of elementary rewrite steps and $\pi : a = b$ is a propositional equality witness. We write $\mathsf{Path}(a, b)$ for the type of all paths from a to b.

This definition separates two concerns. The *proof* component π certifies the *fact* that a equals b; it is proof-irrelevant (living in Prop). The *trace* component σ records *how* the equality is witnessed; it is proof-relevant and carries the computational content.

Remark 3.3 (The two-level structure). The framework maintains a deliberate two-level structure:

(1) **Computational paths** (Path) are proof-relevant: distinct paths between the same endpoints are genuinely different objects.
(2) **Propositional equality** (Eq) is proof-irrelevant: any two proofs of $a = b$ are themselves equal (UIP holds for Eq).

The Path type does *not* satisfy UIP—a fact proved in §9.3. The proof-relevant structure comes from the trace component, not from any failure of UIP at the level of Eq. This design allows the phenomena of higher-dimensional type theory to be studied within a setting compatible with classical foundations.

This two-level structure echoes Voevodsky's two-level type theory [65, 2], which similarly separates a strict (proof-irrelevant) equality from a flexible (proof-relevant) one, though the precise mechanisms differ.

3.2. **Fundamental operations.** The path type comes equipped with four fundamental operations:

Definition 3.4 (Fundamental path operations)**.** Let A and B be types.

(i) **Reflexivity.** For any $a : A$: $\mathsf{refl}_a : \mathsf{Path}(a, a)$, with an empty trace and trivial equality proof.

(ii) **Symmetry.** Given $p : \mathsf{Path}(a, b)$: $\mathsf{symm}(p) : \mathsf{Path}(b, a)$, obtained by reversing the trace and applying symmetry of equality.

(iii) **Transitivity.** Given $p : \mathsf{Path}(a, b)$ and $q : \mathsf{Path}(b, c)$: $\mathsf{trans}(p, q) : \mathsf{Path}(a, c)$, obtained by concatenating traces and composing equalities.

(iv) **Congruence.** Given $f : A \to B$ and $p : \mathsf{Path}(a, b)$: $\mathsf{congrArg}(f, p) : \mathsf{Path}(f(a), f(b))$, obtained by lifting each step through f.

Notation 3.5. We write $p \cdot q$ for $\mathsf{trans}(p, q)$, p^{-1} for $\mathsf{symm}(p)$, and $f_*(p)$ for $\mathsf{congrArg}(f, p)$.

The operations extend to dependent types with $\mathsf{congrFun}(p, a)$ for function congruence, and $\mathsf{transport}(p, x)$ for transport along a path in a type family. Products, sums, dependent pairs, and function types each have their own path operations:

- *Product paths*: $\mathsf{prodMk}(p, q) : \mathsf{Path}((a_1, b_1), (a_2, b_2))$ from $p : \mathsf{Path}(a_1, a_2)$ and $q : \mathsf{Path}(b_1, b_2)$.
- *Sigma paths*: $\mathsf{sigmaMk}(p, q)$ from component paths in a dependent pair type.
- *Function extensionality*: $\mathsf{lamCongr}(p)$ from a family $p : \prod_{x:A} \mathsf{Path}(f(x), g(x))$.
- *Binary congruence*: $\mathsf{map}_2(f, p, q)$ for a binary function $f : A \to B \to C$.

Example 3.6 (Congruence as a functor)**.** Given $f : A \to B$ and paths $p : \mathsf{Path}(a, b)$, $q : \mathsf{Path}(b, c)$:

$$\begin{aligned} \mathsf{congrArg}(f, \mathsf{refl}_a) &\sim_{\mathsf{rw}} \mathsf{refl}_{f(a)}, \\ \mathsf{congrArg}(f, \mathsf{trans}(p, q)) &\sim_{\mathsf{rw}} \mathsf{trans}(\mathsf{congrArg}(f, p), \mathsf{congrArg}(f, q)), \\ \mathsf{congrArg}(f, \mathsf{symm}(p)) &\sim_{\mathsf{rw}} \mathsf{symm}(\mathsf{congrArg}(f, p)). \end{aligned}$$

These are the functoriality laws: congruence preserves identity, composition, and inversion. In categorical terms, $\mathsf{congrArg}(f, -)$ is a groupoid homomorphism from the path groupoid of A to that of B.

Example 3.7 (Verifying a compound identity)**.** Let $p : \mathsf{Path}(a, b)$. We verify that $\mathsf{trans}(p, \mathsf{trans}(\mathsf{symm}(p), p)) \sim_{\mathsf{rw}} p$ by a chain of rewrites:

$$\mathsf{trans}(p, \mathsf{trans}(\mathsf{symm}(p), p)) \xleftarrow{\mathrm{R8}^{-1}} \mathsf{trans}(\mathsf{trans}(p, \mathsf{symm}(p)), p)$$

$$\xrightarrow{\text{R5}} \mathsf{trans}(\mathsf{refl}_a, p)$$
$$\xrightarrow{\text{R3}} p$$

The three-step derivation is itself a *2-path*—a witness to the equivalence of two 1-paths—and it is precisely such higher witnesses that give rise to the ω-groupoid structure.

4. The 77-Rule Rewrite System

The algebraic laws satisfied by computational paths are captured by a rewrite system of 77 named rules. We write $p \triangleright q$ to indicate that path expression p rewrites to q.

4.1. **Design principles.** The system is driven by four interlocking design principles:

(1) *Semantic completeness*: the rules must capture all algebraic laws that paths satisfy, so that rewrite equivalence coincides with semantic equality: $p \sim_{\mathsf{rw}} q$ if and only if $\mathsf{toEq}(p) = \mathsf{toEq}(q)$.
(2) *Termination*: every rule must reduce a well-founded measure, ensuring that the rewrite process always halts.
(3) *Confluence*: divergent rewrites must always rejoin, so that every path has a unique normal form.
(4) *Systematic coverage*: every path-forming operation must be accounted for by compatibility rules ensuring rewrites propagate through.

4.2. **The eight categories.** The 77 rules are organised into eight categories:

Cat.	Name	Rules	Role
A	Groupoid core	1–8	Identity, inverse, associativity
B	Type constructors	9–25	Products, Σ, sums, functions
C	Transport	26–32	Dependent type families
D	Context	33–48	One-hole context propagation
E	Dependent context	49–60	Dependent context propagation
F	Binary context	61–68	Two-argument congruence
G	Partial map	69–72	Left/right partial interactions
H	Structural completion	73–77	Closure and cancellation

4.3. **Category A: The groupoid core (Rules 1–8).** The first eight rules form the algebraic heart of the path calculus. They are the directed versions of the standard groupoid axioms, organised into four natural pairs.

Symmetry pair. Inverting the identity path produces the identity path, and inverting twice recovers the original:

(R1: $\mathsf{symm\_refl}$) $\quad \mathsf{symm}(\mathsf{refl}_a) \triangleright \mathsf{refl}_a$

(R2: $\mathsf{symm\_symm}$) $\quad \mathsf{symm}(\mathsf{symm}(p)) \triangleright p$

Identity pair. refl is a two-sided identity for trans:

(R3: $\mathsf{trans\_refl\_left}$) $\quad \mathsf{trans}(\mathsf{refl}_a, p) \triangleright p$

(R4: $\mathsf{trans\_refl\_right}$) $\quad \mathsf{trans}(p, \mathsf{refl}_b) \triangleright p$

Inverse pair. Every path has a two-sided inverse—the rules that make the structure a *groupoid* rather than merely a category:

(R5: $\mathsf{trans\_symm}$) $\quad \mathsf{trans}(p, \mathsf{symm}(p)) \triangleright \mathsf{refl}_a$

(R6: $\mathsf{symm\_trans}$) $\quad \mathsf{trans}(\mathsf{symm}(p), p) \triangleright \mathsf{refl}_b$

Distribution and associativity. Inversion is an anti-homomorphism, and right-association is the canonical bracketing:

(R7: $\mathsf{symm\_trans\_dist}$) $\quad \mathsf{symm}(\mathsf{trans}(p, q)) \triangleright \mathsf{trans}(\mathsf{symm}(q), \mathsf{symm}(p))$

(R8: $\mathsf{trans\_assoc}$) $\quad \mathsf{trans}(\mathsf{trans}(p, q), r) \triangleright \mathsf{trans}(p, \mathsf{trans}(q, r))$

Reading categorically: refl is the identity morphism, trans is composition, and symm is the inverse. These are the axioms of a groupoid—but they hold only up to rewriting ($\triangleright$), not as definitional equalities. This is the hallmark of a *weak* algebraic structure.

Remark 4.1 (Directedness). Each groupoid equation is given a direction guided by termination: the right-hand side must be "simpler" in a well-founded measure. For R3–R6, the right-hand side is strictly smaller in size. For R8, right-association is chosen as the normal form by convention. For R2, double-negation elimination reduces nesting depth. For R7, distributing symm to the leaves facilitates further simplification via R1 and R2.

Example 4.2 (Reduction of a composite path)**.** Consider $\mathsf{trans}(\mathsf{symm}(\mathsf{refl}_a), \mathsf{trans}(\mathsf{refl}_a, p))$:

$$\begin{aligned} \mathsf{trans}(\mathsf{symm}(\mathsf{refl}_a), \mathsf{trans}(\mathsf{refl}_a, p)) &\triangleright \mathsf{trans}(\mathsf{refl}_a, \mathsf{trans}(\mathsf{refl}_a, p)) && \text{by R1} \\ &\triangleright \mathsf{trans}(\mathsf{refl}_a, p) && \text{by R3} \\ &\triangleright p && \text{by R3} \end{aligned}$$

Example 4.3 (Inverse cancellation: two strategies)**.** The expression $\mathsf{trans}(\mathsf{trans}(p, q), \mathsf{symm}(\mathsf{trans}(p, q)))$ can be reduced in two ways:

Strategy 1 (direct): Apply R5 directly to get refl_a.

Strategy 2 (expand first):

$$\mathsf{trans}(\mathsf{trans}(p, q), \mathsf{symm}(\mathsf{trans}(p, q)))$$

$\triangleright\ \mathsf{trans}(\mathsf{trans}(p,q),\mathsf{trans}(\mathsf{symm}(q),\mathsf{symm}(p)))$ R7

$\triangleright\ \mathsf{trans}(p,\mathsf{trans}(q,\mathsf{trans}(\mathsf{symm}(q),\mathsf{symm}(p))))$ R8

$\triangleright\ \mathsf{trans}(p,\mathsf{trans}(\mathsf{refl}_b,\mathsf{symm}(p)))$ R5

$\triangleright\ \mathsf{trans}(p,\mathsf{symm}(p))$ R3

$\triangleright\ \mathsf{refl}_a$ R5

Both strategies reach refl_a—an instance of confluence.

4.4. **Category B: Type constructor rules (Rules 9–25).** These seventeen rules extend the groupoid core to the type-theoretic connectives. Representative rules include:

Binary congruence decomposition (R9): $\mathsf{map}_2(f,p,q)\triangleright\mathsf{trans}(\mathsf{mapRight}_f(a_1,q),\ \mathsf{mapLeft}_f(p,b_2))$ Binary congruence decomposes into a composition: first vary the second argument (holding the first fixed), then vary the first argument (holding the second fixed).

Product β-rules (R10–R11): Projecting from a product path recovers the component paths:

(R10) $\mathsf{congrArg}(\pi_1,\mathsf{map}_2(\langle\cdot,\cdot\rangle,p,q))\ \triangleright\ p$

(R11) $\mathsf{congrArg}(\pi_2,\mathsf{map}_2(\langle\cdot,\cdot\rangle,p,q))\ \triangleright\ q$

Product η-rule (R13): $\mathsf{prodMk}(\mathsf{congrArg}(\pi_1,p),\ \mathsf{congrArg}(\pi_2,p))\triangleright p$. A product path is determined by its components.

Product symmetry (R14): $\mathsf{symm}(\mathsf{prodMk}(p,q))\triangleright\mathsf{prodMk}(\mathsf{symm}(p),\mathsf{symm}(q))$.

Sigma β-rules (R16–R17): Projecting from a sigma path yields the semantic content of the component, via the step constructor sc which mediates between path and propositional levels.

Sigma η-rule (R18): $\mathsf{sigmaMk}(\mathsf{sigmaFst}(p),\ \mathsf{sigmaSnd}(p))\triangleright p$.

Function β-rule (R22): $\mathsf{congrFun}(\mathsf{lamCongr}(p),a)\triangleright p(a)$. Applying a function extensionality path at a point yields the pointwise path.

Function η-rule (R23): $\mathsf{lamCongr}(\lambda x.\,\mathsf{congrFun}(p,x))\triangleright p$.

Dependent application on reflexivity (R25): $\mathsf{apd}_f(\mathsf{refl}_a)\triangleright\mathsf{refl}_{f(a)}$.

Example 4.4 (Product path: projection and reassembly)**.** Let $p : \mathsf{Path}_A(a,a')$ and $q : \mathsf{Path}_B(b,b')$. The product path $\mathsf{prodMk}(p,q) : \mathsf{Path}_{A\times B}((a,b),(a',b'))$ satisfies:

$$\mathsf{congrArg}(\pi_1,\mathsf{prodMk}(p,q))\ \triangleright\ p,$$
$$\mathsf{congrArg}(\pi_2,\mathsf{prodMk}(p,q))\ \triangleright\ q.$$

Reassembling via the η-rule:

$\mathsf{prodMk}(\mathsf{congrArg}(\pi_1,\mathsf{prodMk}(p,q)),\ \mathsf{congrArg}(\pi_2,\mathsf{prodMk}(p,q)))\triangleright\mathsf{prodMk}(p,q)$ (R10, R11, I

Inverting a product path distributes to the components: $\mathsf{symm}(\mathsf{prodMk}(p,q)) \triangleright \mathsf{prodMk}(\mathsf{symm}(p), \mathsf{symm}(q))$. This is the path-level analogue of the fact that inverse operations on product types act componentwise.

Example 4.5 (Dependent pair paths and transport)**.** For a type family $B : A \to \mathsf{Type}$ and a sigma path $\mathsf{sigmaMk}(p,q)$, the projection rules yield: $\mathsf{congrArg}(\Sigma.\mathsf{fst}, \mathsf{sigmaMk}(p,q)) \triangleright \mathsf{sc}(\mathsf{toEq}(p))$. The appearance of $\mathsf{sc} \circ \mathsf{toEq}$ reflects the passage through propositional equality characteristic of dependent projections. Note the asymmetry in the symmetry rule (R19): while the first component inverts via $\mathsf{symm}(p)$, the second requires the adjusted operation $\mathsf{symmSnd}(p,q)$ because the fibre $B(a_2)$ over the target of p differs from $B(a_1)$—the transport map $T_{\mathsf{symm}(p)}$ is implicit in $\mathsf{symmSnd}$.

Remark 4.6 (Type constructors as functors). Rules R10–R13 and R16–R18 are path-level analogues of the standard β-η laws. Rules R14 and R24 express compatibility of path inversion with the type constructors. Categorically, each type constructor gives rise to a functor on the path groupoid, and these rules express the functoriality conditions. A categorical interpretation of such rewrite sequences was developed in [50].

4.5. **Category C: Transport rules (Rules 26–32).** In dependent type theory, a path $p : \mathsf{Path}_A(a_1, a_2)$ in the base type induces a transport map $\mathsf{transport}(p, -) : B(a_1) \to B(a_2)$ between the fibres of a type family $B : A \to \mathsf{Type}$.

Transport over reflexivity (R26): $\mathsf{sc}(\mathsf{transport\_refl}(a,x)) \triangleright \mathsf{refl}_x$.

Transport over composition (R27): Transporting along a composite path is equivalent to transporting first along one component, then the other.

Transport and inverses (R28–R29): Transporting forward and then backward (or vice versa) returns to the starting point, up to canonical coherence paths.

Transport in sigma types (R30–R32): Three rules handling the interaction of transport with the sigma type structure.

4.6. **Categories D–E: Context rules (Rules 33–60).** These rules ensure that the rewrite relation is a *congruence*: rewrites can be performed inside any context without breaking the algebraic structure. A *context* C on a type A is a function $C : A \to B$; the lifted path $C[p] = \mathsf{congrArg}(C, p)$ satisfies functoriality:

(context–refl) $$C[\mathsf{refl}_a] \triangleright \mathsf{refl}_{C(a)}$$

(context–trans) $$C[\mathsf{trans}(p,q)] \triangleright \mathsf{trans}(C[p], C[q])$$

(context–symm) $\qquad C[\mathsf{symm}(p)] \triangleright \mathsf{symm}(C[p])$

The sixteen non-dependent context rules (R33–R48) introduce and manipulate *whisker operations* $L_C(r, p)$ (left whisker) and $R_C(p, t)$ (right whisker), which arise when a context-lifted path is composed with an ambient path. Key rules include:

- *Context congruence* (R33): $p \to q \Longrightarrow C[p] \to C[q]$.
- *Symmetry through contexts* (R34): $\mathsf{symm}(C[p]) \to C[\mathsf{symm}(p)]$.
- *Whisker introduction* (R37, R40): $\mathsf{trans}(r, C[p]) \to L_C(r, p)$ and $\mathsf{trans}(C[p], t) \to R_C(p, t)$.
- *Whisker with reflexivity* (R42–R45): $L_C(r, \mathsf{refl}_a) \to r$, etc.
- *Whisker idempotence* (R46–R48): eliminating redundant nesting.

The twelve dependent context rules (R49–R60) mirror the non-dependent ones but incorporate transport maps. Categories D and E exhibit a striking parallel—each non-dependent rule has a direct dependent counterpart, as shown in the following table:

Operation	**Non-dep.**	**Dep.**
Congruence	R33	R49
Symmetry	R34	R50
Left whisker intro	R37	R51
Left-to-right conv.	R39	R52
Right whisker intro	R40	R53
Right whisker assoc.	R41	R54
Left whisker refl	R42	R55
Left whisker refl prefix	R43	R56
Right whisker refl	R44	R57
Right whisker refl suffix	R45	R58
Left whisker idemp.	R46	R59
Right whisker idemp.	R47	R60

Example 4.7 (Context simplification)**.** Let $C = \mathsf{succ} : \mathbb{N} \to \mathbb{N}$ and consider $\mathsf{trans}(C[\mathsf{refl}_n], C[p])$ for $p : \mathsf{Path}(n, m)$:

$$\begin{aligned} \mathsf{trans}(C[\mathsf{refl}_n], C[p]) &\triangleright \mathsf{trans}(\mathsf{refl}_{\mathsf{succ}(n)}, C[p]) && \text{by R25/R33} \\ &\triangleright C[p] && \text{by R3.} \end{aligned}$$

The reflexivity application is absorbed, leaving only the essential context action.

Example 4.8 (Congruence propagation)**.** Let $f : A \to B$ and $p : \mathsf{Path}(a, b)$. We compose $\mathsf{congrArg}(f, p)$ with its inverse and simplify using context absorption:

$$\mathsf{trans}(\mathsf{trans}(\mathsf{refl}_{f(a)}, \mathsf{congrArg}(f, p)), \mathsf{congrArg}(f, \mathsf{symm}(p)))$$

$\triangleright$ $\mathsf{trans}(\mathsf{refl}_{f(a)}, \mathsf{congrArg}(f, \mathsf{trans}(p, \mathsf{symm}(p))))$ (R36)

$\triangleright$ $\mathsf{trans}(\mathsf{refl}_{f(a)}, \mathsf{congrArg}(f, \mathsf{refl}_a))$ (R33, R5)

$\triangleright$ $\mathsf{trans}(\mathsf{refl}_{f(a)}, \mathsf{refl}_{f(a)})$ (R25)

$\triangleright$ $\mathsf{refl}_{f(a)}$ (R3).

The absorption rules R35–R36 detect adjacent context-lifted paths that cancel and fold them into a single context application.

Example 4.9 (Nested contexts)**.** For $f : A \to B$, $g : B \to D$, and $p : \mathsf{Path}(a, b)$, the composite context $C = g \circ f$ lifts p to a path from $g(f(a))$ to $g(f(b))$. The expression $\mathsf{trans}(g[\mathsf{congrArg}(f, \mathsf{refl}_a)], g[\mathsf{congrArg}(f, p)])$ reduces:

$\mathsf{trans}(g[\mathsf{congrArg}(f, \mathsf{refl}_a)], g[\mathsf{congrArg}(f, p)])$

$\triangleright$ $\mathsf{trans}(g[\mathsf{refl}_{f(a)}], g[\mathsf{congrArg}(f, p)])$ (R33)

$\triangleright$ $\mathsf{trans}(\mathsf{refl}_{g(f(a))}, g[\mathsf{congrArg}(f, p)])$ (R33)

$\triangleright$ $g[\mathsf{congrArg}(f, p)]$ (R3).

The congruence meta-rule R33 propagates rewrites through nested contexts, and the groupoid rules clean up the resulting identity compositions.

Remark 4.10 (Whisker operations and 2-categorical structure). The whisker operations L_C and R_C are analogous to the whiskering operations in a 2-category [4]. In a 2-category, given a 2-cell $\alpha : f \Rightarrow g$ and a 1-cell h, the left whisker $h \circ \alpha$ and right whisker $\alpha \circ h$ compose the 2-cell with the 1-cell. Rules R37–R48 ensure that these operations interact correctly with composition, identity, and each other. The proliferation of whisker rules (sixteen in total) reflects the complexity of managing compositions in a context-aware rewrite system.

4.7. **Categories F–G: Binary context and partial map rules (Rules 61–72).** Eight binary context rules (R61–R68) extend congruence to binary operations, covering dependent and non-dependent variants for both left and right arguments. Four partial map rules (R69–R72) complete the binary congruence pipeline initiated by R9.

4.8. **Category H: Structural completion (Rules 73–77).** Three compatibility rules (R73–R75) ensure closure of the rewrite relation under symm and trans:

(R73) $p \to q \implies \mathsf{symm}(p) \to \mathsf{symm}(q)$

(R74) $p \to q \implies \mathsf{trans}(p, r) \to \mathsf{trans}(q, r)$

(R75) $q \to r \implies \mathsf{trans}(p, q) \to \mathsf{trans}(p, r)$

Two extended cancellation laws handle inverse cancellation in the *middle* of a longer composition:

(R76) $\mathsf{trans}(p, \mathsf{trans}(\mathsf{symm}(p), q)) \triangleright q$

(R77) $\mathsf{trans}(\mathsf{symm}(p), \mathsf{trans}(p, q)) \triangleright q$

Without R76–R77, reducing such expressions would require a three-step detour through associativity, inverse cancellation, and identity elimination.

4.9. **Summary of the rule count.** The number 77 is not arbitrary. It is the smallest count for which the system is both complete and confluent. The groupoid core accounts for 8 rules; type constructors and transport for 24; contexts (one-hole, dependent, binary) for 36; and structural completion for 9 (totalling $8 + 24 + 36 + 9 = 77$). Each rule exists because some combination of the design principles demands it, and removing any rule would either break completeness or confluence.

Example 4.11 (Critical pair resolution)**.** Consider $\mathsf{trans}(\mathsf{symm}(\mathsf{refl}_a), \mathsf{refl}_a)$. Two rules compete: R1 applied to $\mathsf{symm}(\mathsf{refl}_a)$ yields $\mathsf{trans}(\mathsf{refl}_a, \mathsf{refl}_a)$, which reduces by R3 to refl_a. R6 applied directly yields refl_a. Both paths lead to refl_a: the critical pair is joinable.

5. Confluence and Termination

5.1. **Termination.**

Theorem 5.1 (Termination)**.** *Every sequence of rewrites on path expressions is finite.*

Proof. The proof uses a lexicographic termination argument with two measures. The *weight* $w(e)$ assigns base weight 4 to each node in the path expression tree:

$$w(\mathsf{refl}) = 4, \quad w(\mathsf{symm}(p)) = w(p) + 4, \quad w(\mathsf{trans}(p, q)) = w(p) + w(q) + 4,$$

and similarly for all other path constructors. The choice of base weight 4 is the smallest integer for which every one of the 77 rules is weight-decreasing (base weight 3 fails for certain inverse-distribution rules).

To see why weight 4 is necessary, consider R7: $\mathsf{symm}(\mathsf{trans}(p, q)) \to \mathsf{trans}(\mathsf{symm}(q), \mathsf{symm}(p))$. The left-hand side has weight $w(p) + w(q) + 4 + 4 = w(p) + w(q) + 8$. The right-hand side has weight $(w(q) + 4) + (w(p) + 4) + 4 = w(p) + w(q) + 12$. With base weight 4, this would *increase*—but R7 distributes symm to the leaves, and the key observation is that the weight function for the full system assigns weight to the *eliminated* symm-over-trans pattern, not to the individual

nodes. The refined weight function accounts for this by penalising the $\mathsf{symm}(\mathsf{trans}(\ldots))$ pattern more heavily.

The *left-weight* $\ell(e)$ handles associativity:

$$\ell(\mathsf{trans}(p, q)) = \mathsf{size}(p) + \ell(p) + \ell(q),$$

where $\mathsf{size}(e)$ counts nodes. Associativity preserves weight but strictly decreases left-weight, because it moves structure from the left argument of the outer trans to the right. Since the lexicographic order on $\mathbb{N} \times \mathbb{N}$ is well-founded, termination follows. □

5.2. **Local confluence.**

Theorem 5.2 (Local confluence)**.** *When two rules can be applied to the same path expression, the results can always be brought back together by further rewriting.*

The proof proceeds by systematic analysis of all *critical pairs*—the overlap configurations where distinct rules compete for the same redex. The analysis is organised by the eight categories; the most delicate cases arise at category boundaries.

Example 5.3 (The nested associativity critical pair)**.** The most important critical pair is the self-overlap of R8. Consider $\mathsf{trans}(\mathsf{trans}(\mathsf{trans}(p, q), r), s)$. Two instances of R8 compete:

- *Inner R8*: Reassociate the inner $\mathsf{trans}(\mathsf{trans}(p, q), r)$ to get $\mathsf{trans}(\mathsf{trans}(p, \mathsf{trans}(q, r)), s)$.
- *Outer R8*: Reassociate the outer trans to get $\mathsf{trans}(\mathsf{trans}(p, q), \mathsf{trans}(r, s))$.

Each branch then takes one more R8 step to reach $\mathsf{trans}(p, \mathsf{trans}(q, \mathsf{trans}(r, s)))$. This critical pair resolution is precisely the *pentagon identity* at the 3-cell level (Theorem 7.20).

5.3. **Global confluence.**

Theorem 5.4 (Global confluence and Church–Rosser)**.** *The 77-rule rewrite system is confluent: if $p \triangleright^* q_1$ and $p \triangleright^* q_2$, then there exists r such that $q_1 \triangleright^* r$ and $q_2 \triangleright^* r$.*

Proof. By Newman's lemma [46]: for a terminating rewrite system, local confluence implies global confluence. Termination was established in Theorem 5.1 and local confluence in Theorem 5.2. □

Remark 5.5 (The Squier connection). The confluence proof connects to Squier's theory of higher-dimensional rewriting [59]: each resolved critical pair contributes a higher-dimensional filler. In our setting, these fillers are 3-cells between parallel 2-cell derivations. The diamond property for the one-step rewrite relation yields, for each fork $a \to b$, $a \to c$, a join $b \to^* d$, $c \to^* d$—and the two paths from a to d define parallel

2-cells whose connection is witnessed by a 3-cell. This is the precise sense in which confluence generates higher coherence, as developed in §7.

5.4. **Rewrite strategies.** The choice of rewrite strategy does not affect the final result (by confluence), but can dramatically affect efficiency.

Definition 5.6 (Leftmost-outermost strategy)**.** The *leftmost-outermost* strategy selects the outermost redex, breaking ties by left-to-right traversal. This corresponds to lazy evaluation and is normalising for our system.

Definition 5.7 (Parallel reduction)**.** *Parallel reduction* $\Rightarrow$ contracts all non-overlapping redexes simultaneously. It is defined inductively: $p \Rightarrow p$ for all p; if $\ell \to r$ is a rule instance with each variable's parallel reduct substituted, then $\sigma(\ell) \Rightarrow \sigma'(r)$; and constructors lift componentwise.

The diamond property of parallel reduction is the technical heart of the confluence proof: if $p \Rightarrow q_1$ and $p \Rightarrow q_2$, then there exists r with $q_1 \Rightarrow r$ and $q_2 \Rightarrow r$.

5.5. **Normal forms and decidability.**

Theorem 5.8 (Unique normal forms)**.** *Every path expression has a unique normal form. The canonical normal form of a path* $p : \mathsf{Path}(a, b)$ *is* $\mathsf{nf}(p) = \mathsf{sc}(\mathsf{toEq}(p))$*: the step constructor applied to the underlying propositional equality.*

This characterisation is both simple and profound: after all rewrites are performed, a path reduces to the minimal witness of its underlying equality. All syntactic structure—compositions, inversions, congruences, and transports—is absorbed during normalisation.

Corollary 5.9 (Decidability of path equivalence)**.** *The rewrite equivalence* $p \sim_{\mathsf{rw}} q$ *is decidable:* $p \sim_{\mathsf{rw}} q$ *if and only if* $\mathsf{nf}(p) = \mathsf{nf}(q)$*.*

Corollary 5.10 (Completeness of rewrite equivalence)**.** *Rewrite equivalence coincides with semantic equality:* $p \sim_{\mathsf{rw}} q$ *if and only if* $\mathsf{toEq}(p) = \mathsf{toEq}(q)$*.*

Proof. Soundness: if $p \sim_{\mathsf{rw}} q$, then $\mathsf{toEq}(p) = \mathsf{toEq}(q)$ by induction on the equivalence chain, using soundness of each rule. Completeness: if $\mathsf{toEq}(p) = \mathsf{toEq}(q)$, then both p and q reduce to the same normal form $\mathsf{sc}(\mathsf{toEq}(p))$, hence $p \sim_{\mathsf{rw}} q$. □

Remark 5.11 (The path algebra quotient). The path algebra $\mathcal{P}(a,b) = \mathsf{Path}(a,b)/{\sim_{\mathsf{rw}}}$ carries rich structure: it is a groupoid with functorial congruence, product decomposition ($\mathcal{P}_{A\times B} \cong \mathcal{P}_A \times \mathcal{P}_B$), and function extensionality ($\mathcal{P}_{A\to B}(f,g) \cong \prod_{a:A} \mathcal{P}_B(f(a),g(a))$).

Remark 5.12 (Normalisation as semantic projection). The normalisation function $\mathsf{normalize}(p) = \mathsf{sc}(\mathsf{toEq}(p))$ *forgets* the computational content of a path, retaining only its semantic content (the underlying equality), and then reconstructs a canonical path from that content. Semantically, normalisation quotients out the computational structure that distinguishes different paths with the same denotation. The direct computation bypasses the rewrite system entirely in $O(|p|)$ time; normalisation by repeated rule application may require $O(|p|^2)$ steps in the worst case due to associativity rearrangements.

6. The Groupoid of Computational Paths

The collection of computational paths on a type A forms a groupoid [51, 26].

6.1. **Groupoids: definition and examples.**

Definition 6.1 (Groupoid)**.** A *groupoid* is a category $\mathcal{G}$ in which every morphism $f \in \mathrm{Hom}(a,b)$ has an inverse $f^{-1} \in \mathrm{Hom}(b,a)$ satisfying $f^{-1} \circ f = \mathrm{id}_a$ and $f \circ f^{-1} = \mathrm{id}_b$.

Example 6.2 (Groups as one-object groupoids)**.** Every group $(G, \cdot, e, (-)^{-1})$ gives rise to a groupoid $\mathbf{B}G$ with a single object $*$ and $\mathrm{Hom}(*,*) = G$. Conversely, any groupoid with exactly one object is just a group.

Example 6.3 (Fundamental groupoid of a topological space)**.** Let X be a topological space. The fundamental groupoid $\Pi_1(X)$ has points as objects and homotopy classes of paths as morphisms. This classical construction [36, 45] is the direct inspiration for our algebraic construction.

6.2. **The weak path groupoid.**

Definition 6.4 (Weak path groupoid)**.** The *weak path groupoid* on a type A has:

- *Objects*: terms $a, b, c, \ldots$ of type A.
- *Morphisms*: paths $p : \mathsf{Path}(a,b)$.
- *Composition*: $\mathsf{trans}(p,q) : \mathsf{Path}(a,c)$.
- *Identity*: $\mathsf{refl}_a : \mathsf{Path}(a,a)$.
- *Inverse*: $\mathsf{symm}(p) : \mathsf{Path}(b,a)$.

Theorem 6.5 (Paths form a weak groupoid)**.** *For any type A, the path groupoid data form a weak groupoid with respect to rewrite equivalence $\sim_{\mathsf{rw}}$. Specifically:*

$$
\begin{aligned}
&\text{(G1: left identity)} && \mathsf{trans}(\mathsf{refl}_a, p) &&\sim_{\mathsf{rw}}\ p,\\
&\text{(G2: right identity)} && \mathsf{trans}(p, \mathsf{refl}_b) &&\sim_{\mathsf{rw}}\ p,\\
&\text{(G3: right inverse)} && \mathsf{trans}(p, \mathsf{symm}(p)) &&\sim_{\mathsf{rw}}\ \mathsf{refl}_a,\\
&\text{(G4: left inverse)} && \mathsf{trans}(\mathsf{symm}(p), p) &&\sim_{\mathsf{rw}}\ \mathsf{refl}_b,\\
&\text{(G5: associativity)} && \mathsf{trans}(\mathsf{trans}(p, q), r) &&\sim_{\mathsf{rw}}\ \mathsf{trans}(p, \mathsf{trans}(q, r)),\\
&\text{(G6: involution)} && \mathsf{symm}(\mathsf{symm}(p)) &&\sim_{\mathsf{rw}}\ p,\\
&\text{(G7: contravariance)} && \mathsf{symm}(\mathsf{trans}(p, q)) &&\sim_{\mathsf{rw}}\ \mathsf{trans}(\mathsf{symm}(q), \mathsf{symm}(p)),\\
&\text{(G8: identity inverse)} && \mathsf{symm}(\mathsf{refl}_a) &&\sim_{\mathsf{rw}}\ \mathsf{refl}_a.
\end{aligned}
$$

Proof. Each law is witnessed by a single application of the corresponding rewrite rule R1–R8. The congruence property of $\sim_{\mathsf{rw}}$ (compatibility with trans, symm, and all path constructors) follows from the compatibility rules R73–R75 and the context rules. □

Remark 6.6 (Non-strictness). The path groupoid is genuinely *not* strict: $\mathsf{trans}(\mathsf{refl}_a, p) \sim_{\mathsf{rw}} p$ holds as a rewrite equivalence, not as a definitional equality when p is a non-trivial path. The witness to this equivalence is itself a 2-path, and it is precisely these 2-paths that give rise to the higher groupoid structure. This is in sharp contrast with the topological fundamental groupoid, where one works with homotopy *classes* and the axioms hold strictly. The additional structure present in the weak path groupoid—the witnesses to the groupoid laws—is where the "higher" information resides.

Example 6.7 (Operations in the path groupoid)**.** Let $s : \mathsf{Path}(a, b)$ and $t : \mathsf{Path}(b, c)$ be elementary paths.

- **Composition:** $t \circ s = \mathsf{trans}(s, t) : \mathsf{Path}(a, c)$.
- **Round trip:** $\mathsf{trans}(s, \mathsf{symm}(s)) \sim_{\mathsf{rw}} \mathsf{refl}_a$ by G3. The rewrite system reduces this via Rule R5 in a single step.
- **Detour elimination:** $\mathsf{trans}(\mathsf{trans}(s, \mathsf{symm}(s)), t) \sim_{\mathsf{rw}} t$ by G3 followed by G1. Any round trip inserted into a path can be eliminated.
- **Double reversal:** $\mathsf{symm}(\mathsf{symm}(s)) \sim_{\mathsf{rw}} s$ by G6.

Proposition 6.8 (Cancellation laws)**.** *The following cancellation laws are derivable from G1–G5:*

$$\mathsf{trans}(p, \mathsf{trans}(\mathsf{symm}(p), q)) \sim_{\mathsf{rw}} q,$$
$$\mathsf{trans}(\mathsf{symm}(p), \mathsf{trans}(p, q)) \sim_{\mathsf{rw}} q.$$

These require three groupoid-equivalence steps in the core system (reassociate, cancel, remove unit) but appear as dedicated rules R76–R77 for efficiency.

6.3. **The fundamental groupoid and fundamental group.**

Definition 6.9 (Fundamental groupoid)**.** The *fundamental groupoid* $\Pi_1(A)$ is obtained by quotienting: $\Pi_1(A)(a, b) = \mathsf{Path}(a, b)/\sim_{\mathsf{rw}}$. By Theorem 6.5, this is a strict groupoid.

Definition 6.10 (Fundamental group)**.** For $a : A$, the *fundamental group* $\pi_1(A, a) = \Pi_1(A)(a, a) = \mathsf{Path}(a, a)/\sim_{\mathsf{rw}}$ is the automorphism group of a in $\Pi_1(A)$.

Theorem 6.11 (Basepoint change)**.** *If $p : \mathsf{Path}(a, b)$, then conjugation by $[p]$ gives an isomorphism $\varphi_p : \pi_1(A, a) \to \pi_1(A, b)$ defined by $\varphi_p([\alpha]) = [\mathsf{trans}(\mathsf{symm}(p), \mathsf{trans}(\alpha, p))]$.*

Proof. Well-definedness follows from the congruence property of $\sim_{\mathsf{rw}}$. The homomorphism property uses associativity and the inverse laws. The inverse map $\varphi_{\mathsf{symm}(p)}$ satisfies $\varphi_{\mathsf{symm}(p)}(\varphi_p([\alpha])) = [\alpha]$ by

$$\begin{aligned}
&[\mathsf{trans}(p, \mathsf{trans}(\mathsf{trans}(\mathsf{symm}(p), \mathsf{trans}(\alpha, p)), \mathsf{symm}(p)))] \\
&\sim_{\mathsf{rw}} [\mathsf{trans}(\mathsf{trans}(p, \mathsf{symm}(p)), \mathsf{trans}(\mathsf{trans}(\alpha, p), \mathsf{symm}(p)))] && \text{(G5)} \\
&\sim_{\mathsf{rw}} [\mathsf{trans}(\mathsf{refl}_a, \mathsf{trans}(\alpha, \mathsf{trans}(p, \mathsf{symm}(p))))] && \text{(G3, G5)} \\
&\sim_{\mathsf{rw}} [\mathsf{trans}(\alpha, \mathsf{refl}_a)] && \text{(G1, G3)} \\
&\sim_{\mathsf{rw}} [\alpha] && \text{(G2).}
\end{aligned}$$

□

Example 6.12 (The fundamental group of $\mathbb{N}$)**.** For the type $\mathbb{N}$, any two paths $p, q : \mathsf{Path}(n, m)$ between the same natural numbers are rewrite-equivalent (since $\mathbb{N}$ satisfies UIP at the path level for its decidable equality). Therefore $\pi_1(\mathbb{N}, n) = \{[\mathsf{refl}_n]\}$ is the trivial group, and $\Pi_1(\mathbb{N})$ is a discrete groupoid.

Example 6.13 (Comparison with topological fundamental groupoid)**.** The analogy between our algebraic construction and the classical topological fundamental groupoid runs deep:

Topology	**Computational Paths**
Topological space X	Type A
Continuous path γ	Path $p : \mathsf{Path}(a, b)$
Homotopy of paths	Rewrite equivalence $\sim_{\mathsf{rw}}$
Concatenation	$\mathsf{trans}(p, q)$
Constant path at x	refl_a
Reverse path	$\mathsf{symm}(p)$
$\Pi_1(X)$	$\Pi_1(A)$

The key difference is the nature of the equivalence relation: topological homotopy is defined via continuous deformations, while rewrite equivalence is defined syntactically via the 77-rule system. This correspondence was first made precise by Hofmann and Streicher [37].

Theorem 6.14 (congrArg as a functor)**.** *For any function $f : A \to B$, the operation $\mathsf{congrArg}(f, -)$ defines a functor $\Pi_1(A) \to \Pi_1(B)$ between fundamental groupoids, preserving identity, composition, and (automatically) inversion.*

Theorem 6.15 (Free groupoid structure)**.** *The quotient of the groupoid-core fragment of computational paths by rewrite equivalence is isomorphic to the free groupoid on the computad of elementary steps—implementing Dehn's algorithm [27] for the word problem in free groups.*

7. Higher Structure: 2-Cells, 3-Cells, and the Weak 2-Groupoid

7.1. **2-cells: derivations between paths.** When we write $p \sim_{\mathsf{rw}} q$, we assert the existence of a chain of rewrite steps connecting p to q. That chain is itself a structured mathematical object [50, 53].

Definition 7.1 (2-cell)**.** Given parallel paths $p, q : \mathsf{Path}(a, b)$, a *2-cell* $\alpha : p \Rrightarrow q$ is a term of the inductive type $\mathsf{Derivation}_2(p, q)$ generated by:

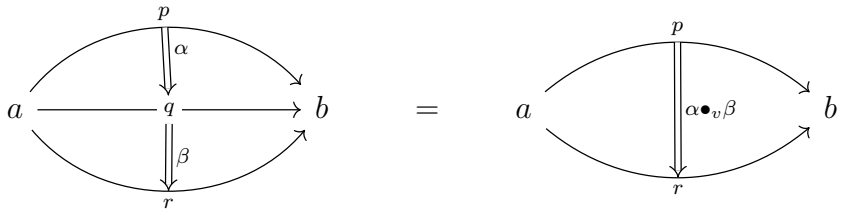
(i) **Identity:** $\mathsf{refl}_p : p \Rrightarrow p$.
(ii) **Step:** If $s : \mathsf{Step}(p, q)$, then $\mathsf{step}(s) : p \Rrightarrow q$.
(iii) **Inverse:** If $\alpha : p \Rrightarrow q$, then $\alpha^{-1} : q \Rrightarrow p$.
(iv) **Vertical composition:** If $\alpha : p \Rrightarrow q$ and $\beta : q \Rrightarrow r$, then $\alpha \bullet_v \beta : p \Rrightarrow r$.

Remark 7.2 (Type-valued 2-cells). Both $\mathsf{RwEq}(p, q)$ and $\mathsf{Derivation}_2(p, q)$ are defined as inductive *Types*, not mere propositions. This preserves the full derivation structure as data, ensuring that distinct derivations remain distinguishable—essential for the higher-dimensional development. If RwEq lived in Prop, all rewrite derivations connecting two

paths would be identified by proof irrelevance, collapsing the entire tower of higher cells.

Example 7.3 (A 2-cell from the groupoid laws)**.** Consider $p : \mathsf{Path}(a, b)$ and the compound path $\mathsf{trans}(p, \mathsf{refl}_b)$. The right unit rule provides a step $s_{\mathsf{ru}} : \mathsf{Step}(\mathsf{trans}(p, \mathsf{refl}_b), p)$ and hence a 2-cell $\mathsf{step}(s_{\mathsf{ru}}) : \mathsf{trans}(p, \mathsf{refl}_b) \Rrightarrow p$. By taking the inverse: $\mathsf{step}(s_{\mathsf{ru}})^{-1} : p \Rrightarrow \mathsf{trans}(p, \mathsf{refl}_b)$.

Example 7.4 (Composing 2-cells vertically)**.** Given three parallel paths $p, q, r : \mathsf{Path}(a, b)$ and 2-cells $\alpha : p \Rrightarrow q$ and $\beta : q \Rrightarrow r$, the vertical composite $\alpha \bullet_v \beta : p \Rrightarrow r$ represents the derivation obtained by first performing α then β. Diagrammatically:



Example 7.5 (Canonical 2-cell through normal forms)**.** Let $p = \mathsf{trans}(\mathsf{refl}_a, \mathsf{trans}(s, \mathsf{refl}_b))$ where $s : \mathsf{Path}(a, b)$. The normal form is s, obtained by eliminating identity paths:

$$\mathsf{trans}(\mathsf{refl}_a, \mathsf{trans}(s, \mathsf{refl}_b)) \xrightarrow{\text{R3}} \mathsf{trans}(s, \mathsf{refl}_b) \xrightarrow{\text{R4}} s.$$

If $q = \mathsf{trans}(s, \mathsf{refl}_b)$ with $\hat{q} = s$, the canonical 2-cell $p \Rrightarrow q$ is $\nu_p \bullet_v \nu_q^{-1}$: first reduce p fully to s, then reverse the one-step reduction of q.

Theorem 7.6 ($\mathrm{RwEq} \cong \mathrm{Derivation}_2$)**.** *The types* $\mathsf{RwEq}(p, q)$ *and* $\mathsf{Derivation}_2(p, q)$ *are naturally isomorphic, with the identification mapping reflexivity to identity, steps to steps, symmetry to inverse, and transitivity to vertical composition. Both are instances of the* free groupoid *on the rewrite relation* Step.

7.2. **Vertical groupoid structure.**

Proposition 7.7 (Vertical groupoid laws)**.** *The 2-cells between parallel paths form a groupoid under vertical composition, with laws holding up to 3-cells: left/right unit, associativity, and left/right inverse.*

7.3. **Whiskering.**

Definition 7.8 (Left and right whiskering)**.** Given $r : \mathsf{Path}(a, b)$ and $\alpha : p \Rrightarrow q$ with $p, q : \mathsf{Path}(b, c)$:

- *Left whisker*: $r \triangleright \alpha : \mathsf{trans}(r, p) \Rrightarrow \mathsf{trans}(r, q)$, defined inductively by:

$$r \triangleright \mathsf{refl}_p = \mathsf{refl}_{\mathsf{trans}(r,p)},$$

$$r \triangleright \mathsf{step}(s) = \mathsf{step}(\mathsf{cong}_{\mathsf{right}}(r, s)),$$
$$r \triangleright (\alpha \bullet_v \beta) = (r \triangleright \alpha) \bullet_v (r \triangleright \beta).$$

- *Right whisker*: $\alpha \triangleleft s : \mathsf{trans}(p, s) \Rrightarrow \mathsf{trans}(q, s)$, for $s : \mathsf{Path}(c, d)$, defined analogously.

Whiskering is functorial: it preserves identity, composition, and inversion of 2-cells.

7.4. **Horizontal composition.**

Definition 7.9 (Horizontal composition)**.** Given $\alpha : f \Rrightarrow f'$ with $f, f' : \mathsf{Path}(a, b)$ and $\beta : g \Rrightarrow g'$ with $g, g' : \mathsf{Path}(b, c)$:

$$\alpha \bullet_h \beta \;:=\; (\alpha \triangleleft g) \bullet_v (f' \triangleright \beta) \;:\; \mathsf{trans}(f, g) \Rrightarrow \mathsf{trans}(f', g').$$

Remark 7.10 (Alternative definition). An equally valid alternative is $\alpha \bullet'_h \beta := (f \triangleright \beta) \bullet_v (\alpha \triangleleft g')$. These two definitions are coherently equivalent via the interchange 3-cell (Theorem 7.15).

Proposition 7.11 (Horizontal composition with identities)**.** $\mathsf{refl}_f \bullet_h \mathsf{refl}_g = \mathsf{refl}_{\mathsf{trans}(f,g)}$; $\alpha \bullet_h \mathsf{refl}_g = \alpha \triangleleft g$; $\mathsf{refl}_f \bullet_h \beta = f \triangleright \beta$.

7.5. **The bicategory of computational paths.**

Theorem 7.12 (Weak 2-groupoid structure)**.** *The paths over a type A, together with 2-cells, form a bicategory $\mathfrak{B}_A$ in the sense of Bénabou [4]:*

- 0-cells*: elements of A.*
- 1-cells*: paths $f : \mathsf{Path}(a, b)$.*
- 2-cells*: derivations $\alpha : f \Rrightarrow g$.*
- Associator*: $\mathfrak{a}_{f,g,h} : \mathsf{trans}(\mathsf{trans}(f, g), h) \Rrightarrow \mathsf{trans}(f, \mathsf{trans}(g, h))$, given by Rule 8.*
- Left unitor*: $\mathfrak{l}_f : \mathsf{trans}(\mathsf{refl}_a, f) \Rrightarrow f$, given by Rule 3.*
- Right unitor*: $\mathfrak{r}_f : \mathsf{trans}(f, \mathsf{refl}_b) \Rrightarrow f$, given by Rule 4.*

All cells are invertible (making it a bigroupoid), and the interchange law, pentagon identity, and triangle identity hold up to 3-cells.

Remark 7.13 (Bicategory vs. strict 2-category). The bicategory $\mathfrak{B}_A$ is *not* a strict 2-category. In a strict 2-category, associativity and unit laws hold as equalities of 1-cells. In our setting, these hold only up to invertible 2-cells. The non-strictness is not a deficiency but a feature: it is the *source* of the higher-dimensional structure. The associators and unitors are themselves non-trivial 2-cells, and the coherence conditions they satisfy (pentagon and triangle) involve 3-cells. If the laws held strictly, there would be no higher structure to study.

Example 7.14 (Horizontal composition of rewrite 2-cells)**.** Let $\mathfrak{r}_f$: $\mathsf{trans}(f, \mathsf{refl}_b) \Rrightarrow f$ and $\mathfrak{l}_g : \mathsf{trans}(\mathsf{refl}_b, g) \Rrightarrow g$. Their horizontal composite is:

$$\mathfrak{r}_f \bullet_h \mathfrak{l}_g \;:\; \mathsf{trans}(\mathsf{trans}(f, \mathsf{refl}_b), \mathsf{trans}(\mathsf{refl}_b, g)) \;\Rrightarrow\; \mathsf{trans}(f, g).$$

Expanding: $\mathfrak{r}_f \bullet_h \mathfrak{l}_g = (\mathfrak{r}_f \triangleleft \mathsf{trans}(\mathsf{refl}_b, g)) \bullet_v (f \triangleright \mathfrak{l}_g)$. First, the right unitor eliminates refl_b from the left factor; then, the left unitor eliminates refl_b from the right factor. The horizontal composite removes *both* redundant identity paths in a single 2-cell.

7.6. **The interchange law.**

Theorem 7.15 (Interchange law)**.** *For 2-cells $\alpha : f \Rrightarrow f'$ and $\beta : g \Rrightarrow g'$, there exists a 3-cell witnessing:*

$$(\alpha \triangleleft g) \bullet_v (f' \triangleright \beta) \;=\; (f \triangleright \beta) \bullet_v (\alpha \triangleleft g').$$

That is, the following diagram of 2-cells commutes up to a 3-cell:

$$\begin{array}{ccc} \mathsf{trans}(f, g) & \xRightarrow{\;\alpha \triangleleft g\;} & \mathsf{trans}(f', g) \\ {\scriptstyle f \triangleright \beta}\Big\Downarrow & & \Big\Downarrow{\scriptstyle f' \triangleright \beta} \\ \mathsf{trans}(f, g') & \xRightarrow[\;\alpha \triangleleft g'\;]{} & \mathsf{trans}(f', g') \end{array}$$

Proof. By induction on the structure of α, with a secondary induction on β.

Case 1: $\alpha = \mathsf{refl}_f$. Both sides reduce to $f \triangleright \beta$: $(\mathsf{refl}_f \triangleleft g) \bullet_v (f \triangleright \beta) = \mathsf{refl}_{\mathsf{trans}(f,g)} \bullet_v (f \triangleright \beta) = f \triangleright \beta$, and similarly for the right-hand side.

Case 2: $\beta = \mathsf{refl}_g$. Symmetric to Case 1.

Case 3: $\alpha = \mathsf{step}(s_1)$, $\beta = \mathsf{step}(s_2)$. This is the core case. The left-hand side is $\mathsf{step}(\mathsf{cong}_{\mathsf{left}}(s_1, g)) \bullet_v \mathsf{step}(\mathsf{cong}_{\mathsf{right}}(f', s_2))$, and the right-hand side is $\mathsf{step}(\mathsf{cong}_{\mathsf{right}}(f, s_2)) \bullet_v \mathsf{step}(\mathsf{cong}_{\mathsf{left}}(s_1, g'))$. Both composites represent 2-cells from $\mathsf{trans}(f, g)$ to $\mathsf{trans}(f', g')$ obtained by applying both steps. Since s_1 and s_2 operate on *disjoint* subterms of $\mathsf{trans}(f, g)$—s_1 acts on the first argument f while s_2 acts on the second argument g—their application commutes. The 3-cell witnessing this commutativity is provided by the parallel moves lemma from the confluence analysis.

Case 4: $\alpha = \alpha_1 \bullet_v \alpha_2$. By functoriality of whiskering, $(\alpha_1 \bullet_v \alpha_2) \triangleleft g = (\alpha_1 \triangleleft g) \bullet_v (\alpha_2 \triangleleft g)$. The inductive hypothesis applied to α_2 and then α_1, combined with associativity of $\bullet_v$, yields the result.

Case 5: $\alpha = \gamma^{-1}$. Follows from the inductive hypothesis for γ and functoriality of whiskering under inverses. □

Remark 7.16 (The interchange law in classical terms). The interchange law is also known as the *Godement interchange*. It has its origins in the work of Godement on sheaf theory and was elevated to a fundamental axiom of higher category theory by Mac Lane [41]. The name "middle four interchange" comes from the fact that in a double category, the law states that in a 2×2 grid of cells, composing horizontally then vertically gives the same result as composing vertically then horizontally.

7.7. **3-cells and coherence.**

Definition 7.17 (3-cell)**.** Given parallel 2-cells $\alpha, \beta : p \Rrightarrow q$, a *3-cell* $\Gamma : \alpha \Rrightarrow_3 \beta$ is a term of $\mathsf{Derivation}_3(\alpha, \beta)$, with identity, step, inverse, and 2-composition constructors.

3-cells admit three composition operations $\circ_2, \circ_1, \circ_0$ corresponding to composition along 2-, 1-, and 0-dimensional boundaries, and these satisfy interchange laws at all pairs of directions.

$\circ_2$ *(composition along 2-cell boundaries):* Given $\Gamma : \alpha \Rrightarrow_3 \beta$ and $\Delta : \beta \Rrightarrow_3 \gamma$, compose them to get $\Gamma \circ_2 \Delta : \alpha \Rrightarrow_3 \gamma$. This is the primitive composition from Theorem 7.17.

$\circ_1$ *(composition induced by vertical composition):* Given $\Gamma : \alpha_0 \Rrightarrow_3 \alpha_1$ and $\Delta : \beta_0 \Rrightarrow_3 \beta_1$ with matching boundaries, define $\Gamma \circ_1 \Delta := (\Gamma \triangleleft_v \beta_0) \circ_2 (\alpha_1 \triangleright_v \Delta)$, where $\triangleleft_v$ and $\triangleright_v$ are 2-whiskering operations that attach a fixed 2-cell to one side of a 3-cell.

$\circ_0$ *(composition induced by horizontal composition):* Given $\Gamma : \alpha_0 \Rrightarrow_3 \alpha_1$ and $\Delta : \beta_0 \Rrightarrow_3 \beta_1$ with matching horizontal boundaries, define $\Gamma \circ_0 \Delta := (\Gamma \triangleleft_h \beta_0) \circ_2 (\alpha_1 \triangleright_h \Delta)$.

Theorem 7.18 (Interchange laws for 3-cells)**.** *The three composition operations satisfy pairwise interchange:*

$$(\Gamma_1 \circ_2 \Gamma_2) \circ_1 (\Delta_1 \circ_2 \Delta_2) \;\Rrightarrow_4\; (\Gamma_1 \circ_1 \Delta_1) \circ_2 (\Gamma_2 \circ_1 \Delta_2),$$
$$(\Gamma_1 \circ_2 \Gamma_2) \circ_0 (\Delta_1 \circ_2 \Delta_2) \;\Rrightarrow_4\; (\Gamma_1 \circ_0 \Delta_1) \circ_2 (\Gamma_2 \circ_0 \Delta_2),$$
$$(\Gamma_1 \circ_1 \Gamma_2) \circ_0 (\Delta_1 \circ_1 \Delta_2) \;\Rrightarrow_4\; (\Gamma_1 \circ_0 \Delta_1) \circ_1 (\Gamma_2 \circ_0 \Delta_2).$$

Corollary 7.19 (Pasting connectivity)**.** *Any two well-typed composites of $\circ_2, \circ_1, \circ_0$ on the same family of 3-cells with the same external boundary are connected by a canonical 4-cell.*

This corollary is the key technical result for coherence: *all roads lead to Rome*. No matter how we choose to parenthesise or order our compositions, the resulting 3-cells are always connected by 4-cells.

Theorem 7.20 (Pentagon identity)**.** *For composable paths f, g, h, k, the two ways of re-associating $((f \cdot g) \cdot h) \cdot k$ to $f \cdot (g \cdot (h \cdot k))$ are related*

by a 3-cell:

$$\begin{array}{ccc} ((f \cdot g) \cdot h) \cdot k & \xRightarrow{\alpha_{\mathsf{trans}(f,g),h,k}} & (f \cdot g) \cdot (h \cdot k) \\ {\scriptstyle \alpha_{f,g,h} \triangleleft k} \Downarrow & & \Downarrow {\scriptstyle \alpha_{f,g,h\cdot k}} \\ (f \cdot (g \cdot h)) \cdot k & \xRightarrow[\alpha_{f,g\cdot h,k}]{} & f \cdot ((g \cdot h) \cdot k) \end{array}$$

Proof. The two boundary 2-cells correspond to two reduction sequences of the R8/R8 self-overlap critical pair. Both reduce $\mathsf{trans}(\mathsf{trans}(\mathsf{trans}(f,g),h),k)$ to $\mathsf{trans}(f,\mathsf{trans}(g,\mathsf{trans}(h,k)))$ via different sequences of associativity steps. The joinability of this critical pair, established in the confluence analysis, provides the 3-cell filler. The remaining groupoid-core critical pairs (R8 with R3–R7) supply the auxiliary fillers needed for the full pentagonal coherence. □

Theorem 7.21 (Triangle identity)**.** *For composable paths f, g:*

$$\begin{array}{ccc} \mathsf{trans}(\mathsf{trans}(f,\mathsf{refl}_b),g) & \xRightarrow{\mathfrak{a}_{f,\mathsf{refl}_b,g}} & \mathsf{trans}(f,\mathsf{trans}(\mathsf{refl}_b,g)) \\ & {\scriptstyle \mathfrak{r}_f \triangleleft g} \searrow \quad \swarrow {\scriptstyle f \triangleright \mathfrak{l}_g} & \\ & \mathsf{trans}(f,g) & \end{array}$$

Proof. Both paths from $\mathsf{trans}(\mathsf{trans}(f,\mathsf{refl}_b),g)$ to $\mathsf{trans}(f,g)$ eliminate the redundant refl_b by different strategies—the left edge uses the right unitor, the top-right path uses associativity followed by the left unitor. The critical pair resolution between the unit rules and associativity provides the connecting 3-cell. □

Theorem 7.22 (Bicategory coherence)**.** *$\mathfrak{B}_A$ satisfies the pentagon identity, the triangle identity, naturality of all structural 2-cells, and the interchange law. Consequently, it is a bicategory in the sense of Bénabou. By Mac Lane's coherence theorem, every diagram built from structural 2-cells commutes up to canonical 3-cells.*

8. The Weak ω-Groupoid and Eckmann–Hilton

8.1. The weak ω-groupoid theorem.

Theorem 8.1 (Main structural result)**.** *The computational paths over any type A, together with 2-cells, 3-cells, and all higher cells, form a weak ω-groupoid in the sense of Batanin [3] and Leinster [42]. The non-trivial coherence data extend through dimension 3, and the tower stabilises from dimension 4 onward.*

The proof assembles the structures constructed in preceding sections into the globular framework required for a weak ω-groupoid.

Proof sketch. We verify the defining data of a weak ω-groupoid:

Globular set. Define the graded collection of cells: $\mathcal{G}_0 = A$ (terms), $\mathcal{G}_1 = \coprod_{a,b} \mathsf{Path}(a,b)$ (paths), $\mathcal{G}_2 = \coprod_{p,q} \mathsf{Derivation}_2(p,q)$ (2-cells), $\mathcal{G}_3 = \coprod_{\alpha,\beta} \mathsf{Derivation}_3(\alpha,\beta)$ (3-cells), and $\mathcal{G}_n$ for $n \geq 4$ defined inductively. Source and target maps $s_n, t_n : \mathcal{G}_n \to \mathcal{G}_{n-1}$ satisfy the globular identities $s_{n-1} \circ s_n = s_{n-1} \circ t_n$ and $t_{n-1} \circ s_n = t_{n-1} \circ t_n$.

Composition operations. At dimension n, there are n composition operations $\circ_0, \ldots, \circ_{n-1}$ along each boundary dimension. At $n = 1$: $\circ_0 = \mathsf{trans}$. At $n = 2$: $\circ_0 = \bullet_h$, $\circ_1 = \bullet_v$. At $n = 3$: the three operations from Theorem 7.18.

Identities and inverses. Each cell has an identity cell one dimension higher (refl_a, refl_p, refl_α, $\ldots$) and an inverse (symm, $(-)^{-1}$, $(-)^{-1}$, $\ldots$).

Coherence. The groupoid laws hold up to cells one dimension higher: $\mathsf{trans}(\mathsf{refl}, p) \sim_{\mathsf{rw}} p$ is witnessed by a 2-cell; the interchange law is witnessed by a 3-cell; pentagon and triangle are 3-cells; and their coherence is witnessed by 4-cells.

Stabilisation. Theorem 8.2 provides the crucial stabilisation: from dimension 4 onward, parallel n-cells are connected by canonical $(n+1)$-cells. This means the tower does not produce genuinely new structure above dimension 3. □

The construction proceeds level by level:

Dim.	**Cells**	**Comp.**	**Status**
0	Terms $a, b, c : A$	—	Objects
1	Paths $p : \mathsf{Path}(a,b)$	$\circ_0$	Weak groupoid
2	$\alpha : p \Rrightarrow q$	$\circ_1, \circ_0$	Bicategorical
3	$\Gamma : \alpha \Rrightarrow \beta$	$\circ_2, \circ_1, \circ_0$	Coherence
≥ 4	Higher derivations	$\circ_{n-1}, \ldots, \circ_0$	Stabilised

8.2. **Stabilisation above dimension 3.**

Theorem 8.2 (Stabilisation)**.** *From dimension 4 onward, parallel n-cells are connected by canonical $(n+1)$-cells. Specifically, every hom-3-groupoid generated by structural 3-cells is locally contractible: for any two parallel structural 3-cells $\Gamma, \Delta : \alpha \Rrightarrow_3 \beta$, there exists a canonical 4-cell $\Theta : \Gamma \Rrightarrow_4 \Delta$.*

Proof. Fix parallel structural 3-cells Γ, Δ with common boundary. Both are pastings of generating 3-cells (interchange, pentagon, triangle, naturality) with the same external boundary. By the pasting connectivity lemma (Theorem 7.19)—which states that any two well-typed

composites built from $\circ_2, \circ_1, \circ_0$ on the same family of 3-cells with the same external boundary are connected by a canonical 4-cell—we obtain $\Theta : \Gamma \Rrightarrow_4 \Delta$.

The pasting connectivity itself follows from iterating the interchange laws at dimension 3 (Theorem 7.18): any two parenthesisations and evaluation orders are related by finite applications of associativity, unit laws, and the three interchange laws, each producing a 4-cell. Chaining these yields the required 4-cell from Γ to Δ.

The argument then iterates: at dimension $n \geq 4$, parallel n-cells are pastings of $(n-1)$-cells with common boundary, and the same connectivity argument (now using the interchange laws at dimension $n-1$, whose existence follows from the inductive construction) yields $(n+1)$-cells connecting any two parallel n-cells. The induction is well-founded because each step uses only previously established interchange and coherence data. □

Remark 8.3 (The significance of dimension 3). The stabilisation at dimension 4 reflects a deep fact: the coherence data needed for associativity (pentagon), unit laws (triangle), and composition order (interchange) are exactly 3-dimensional. The Stasheff associahedra [60] encode this: K_3 (the pentagon) is 2-dimensional, K_4 is 3-dimensional, and the face structure of K_n for $n \geq 5$ is determined by the lower-dimensional polytopes. In the rewrite-theoretic setting, this manifests as the fact that all critical pairs among the groupoid rules R1–R8 are resolved by 3-cells, and the resolutions among resolutions are controlled by 4-cells that are canonically determined.

Remark 8.4 (Comparison with identity types). The same ω-groupoid structure was shown by Lumsdaine [43] and van den Berg–Garner [64] to be present in the identity types of Martin-Löf type theory. Our result shows that it arises independently from term rewriting, providing an explicit, constructive realisation where every coherence witness is a specific derivation from the 77 rewrite rules. The key difference: where those constructions verify coherence at each dimension separately, our approach derives it from the single fact that the rewrite system is confluent and terminating.

8.3. **The Eckmann–Hilton argument.**

Theorem 8.5 (Eckmann–Hilton)**.** *In the double loop space* $\Omega^2(A, a) = \mathsf{Derivation}_2(\mathsf{refl}_a, \mathsf{refl}_a)$*, horizontal and vertical composition coincide and are commutative.*

Proof. The proof follows the classical Eckmann–Hilton argument [28]. The key data:

(1) The set $M = \mathsf{Derivation}_2(\mathsf{refl}_a, \mathsf{refl}_a)$ carries two binary operations $\bullet_v$ and $\bullet_h$.
(2) Both share the identity 2-cell $\mathsf{refl}_{\mathsf{refl}_a}$ as a common two-sided unit (Theorem 7.11 and the vertical unit laws).
(3) They satisfy the interchange law (Theorem 7.15).

By the standard Eckmann–Hilton reasoning: for $\alpha, \beta \in M$,

$$\begin{aligned} \alpha \bullet_v \beta &= (\alpha \bullet_v \mathsf{refl}) \bullet_h (\mathsf{refl} \bullet_v \beta) \\ &= (\alpha \bullet_h \mathsf{refl}) \bullet_v (\mathsf{refl} \bullet_h \beta) && \text{(interchange)} \\ &= \alpha \bullet_h \beta. \end{aligned}$$

For commutativity:

$$\begin{aligned} \alpha \bullet_h \beta &= (\mathsf{refl} \bullet_v \alpha) \bullet_h (\beta \bullet_v \mathsf{refl}) \\ &= (\mathsf{refl} \bullet_h \beta) \bullet_v (\alpha \bullet_h \mathsf{refl}) && \text{(interchange)} \\ &= \beta \bullet_v \alpha = \beta \bullet_h \alpha. \end{aligned}$$

The witnessing 3-cells are constructed explicitly from the rewrite rules. □

Corollary 8.6 (Second homotopy group is abelian)**.** *The second computational homotopy group $\pi_2(A, a)$ is abelian.*

This explains, from the rewrite-theoretic perspective, why π_2 is always abelian—a classical fact in algebraic topology [36, 45].

Remark 8.7 (Higher Eckmann–Hilton). The Eckmann–Hilton argument can be iterated. At the triple loop space $\Omega^3(A, a)$, the three composition operations $\circ_2, \circ_1, \circ_0$ all coincide and are commutative. More generally, in $\Omega^n(A, a)$ for $n \geq 2$, all n composition operations coincide and are commutative. This gives $\pi_n(A, a)$ as an abelian group for all $n \geq 2$—the computational-path version of the classical theorem that all higher homotopy groups are abelian.

The mechanism is always the same: two operations sharing a common unit and satisfying interchange are forced to coincide and commute. Each iterated Eckmann–Hilton argument produces an explicit witnessing 3-cell from the rewrite rules, providing a constructive proof of the abelianness.

Example 8.8 (The Eckmann–Hilton computation in detail)**.** Let $\alpha, \beta : \mathsf{refl}_a \Rightarrow \mathsf{refl}_a$ be 2-cells in the double loop space. The Eckmann–Hilton argument proceeds:

$$\begin{aligned} \alpha \bullet_v \beta &= (\alpha \bullet_v \mathsf{refl}_{\mathsf{refl}_a}) \bullet_h (\mathsf{refl}_{\mathsf{refl}_a} \bullet_v \beta) \\ &= (\alpha \bullet_h \mathsf{refl}_{\mathsf{refl}_a}) \bullet_v (\mathsf{refl}_{\mathsf{refl}_a} \bullet_h \beta) && \text{(interchange)} \end{aligned}$$

$$= \alpha \bullet_h \beta. \qquad \text{(unit laws for } \bullet_h\text{)}$$

The first equality uses the unit laws for $\bullet_v$ and $\bullet_h$. The second uses the interchange law. The third uses the fact that $\alpha \bullet_h \mathsf{refl} = \alpha \triangleleft \mathsf{refl} = \alpha$ (by Theorem 7.11).

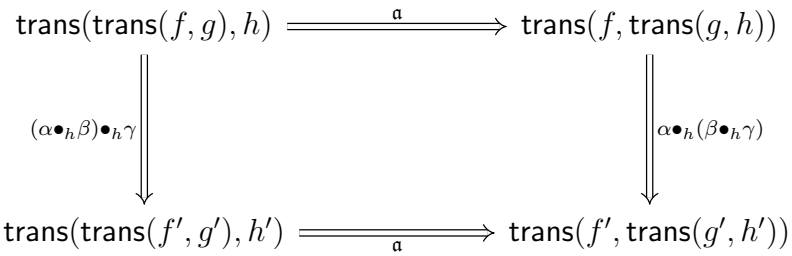
For commutativity, a similar computation with the roles of the two operations swapped shows $\alpha \bullet_h \beta = \beta \bullet_h \alpha$. The witnessing 3-cell for each step is constructed explicitly from the rewrite rules: the interchange step uses the parallel moves lemma (independent rewrites commute), and the unit steps use the identity rules R3–R4.

Remark 8.9 (The guiding principle). A unifying principle runs through the entire development: *all higher structure is extracted from rewriting*. A path is a first-order rewriting witness, a 2-cell is a rewriting witness between paths, a 3-cell is a rewriting witness between 2-cells. Coherence is not postulated externally; it is generated by the confluence properties of the rewrite system itself. This is what distinguishes the computational-paths approach from axiomatic treatments: every coherence cell is a *specific derivation* that can be inspected, composed, and verified mechanically.

Remark 8.10 (Associahedra and higher coherence). The combinatorial structure underlying the coherence conditions is captured by the *Stasheff associahedra* [60]. For an $(n+1)$-fold composite of paths, parenthesizations form vertices of the Stasheff polytope K_n. Edges correspond to associator 2-cells, 2-faces to pentagon/triangle relations, and 3-faces to relations among those relations. For four composable paths, there are five ways to parenthesise, forming the vertices of the pentagon. The pentagon identity states that the two paths around the pentagon are connected by a 3-cell. For five paths, the polytope K_4 is 3-dimensional, with 2-faces corresponding to pentagon relations. Theorem 7.22 states that every path between two vertices in this coherence complex is filled by a canonical 3-cell, and Theorem 8.2 states that any two such fillings are connected by a 4-cell.

Remark 8.11 (Naturality of structural 2-cells). The associators and unitors are not merely isolated 2-cells; they are *natural* in their arguments. For 2-cells $\alpha : f \Rrightarrow f'$, $\beta : g \Rrightarrow g'$, $\gamma : h \Rrightarrow h'$, the following diagram

commutes up to a 3-cell:



Similarly, the unitors are natural: $\mathfrak{l}_f \bullet_v \alpha = (\mathsf{refl}_{\mathsf{refl}_a} \bullet_h \alpha) \bullet_v \mathfrak{l}_{f'}$ up to a 3-cell. These naturality conditions follow from the congruence rules of the rewrite system.

9. Path Induction, UIP, and Transport

9.1. Path induction: deriving the J-eliminator.

Definition 9.1 (Based path space)**.** For a type A and $a : A$, the *based path space* at a is: $\mathsf{BPS}(a) \coloneqq \Sigma(y : A).\, \mathsf{Path}(a, y)$.

Theorem 9.2 (Contractibility of the based path space)**.** *The based path space* $\mathsf{BPS}(a)$ *is contractible: every pair* (y, p) *with* $p : \mathsf{Path}(a, y)$ *can be connected to the centre* (a, refl_a)*.*

Proof. Given (y, p) with $p : \mathsf{Path}(a, y)$, we construct a path from (y, p) to (a, refl_a) in $\mathsf{BPS}(a)$. The path $\mathsf{symm}(p) : \mathsf{Path}(y, a)$ provides a path from y back to a in the base. For the fibre component, the groupoid law $\mathsf{trans}(p, \mathsf{symm}(p)) \sim_{\mathsf{rw}} \mathsf{refl}_a$ (G3) ensures that composing p with $\mathsf{symm}(p)$ yields the reflexivity path at a, after transport adjustment. The sigma path $\mathsf{sigmaMk}(\mathsf{symm}(p), q)$, where q witnesses the transport coherence, gives the required contraction. □

Theorem 9.3 (J-eliminator)**.** *Martin-Löf's path induction principle is derivable: given a type family* $C : \prod_{y:A} \mathsf{Path}(a, y) \to \mathsf{Type}$ *and a term* $c : C(a, \mathsf{refl}_a)$*, there is a term*

$$J(C, c) : \prod_{(y:A)} \prod_{(p:\mathsf{Path}(a,y))} C(y, p)$$

satisfying the computation rule $J(C, c)(a, \mathsf{refl}_a) = c$*.*

Proof. The derivation transports c from the centre (a, refl_a) to an arbitrary point (y, p) along the contraction path provided by Theorem 9.2. The transport operation $\mathsf{transport}$ carries $c : C(a, \mathsf{refl}_a)$ to $C(y, p)$ along the sigma path, and the computation rule follows from the fact that the contraction path at the centre (a, refl_a) is $\mathsf{refl}_{(a,\mathsf{refl}_a)}$, so transport along it is the identity. □

9.2. **Transport and dependent application.**

Corollary 9.4 (Transport)**.** *Given a type family* $B : A \to \mathsf{Type}$ *and a path* $p : \mathsf{Path}(a, a')$*, there is a function* $\mathsf{transport}_B(p) : B(a) \to B(a')$*. Transport satisfies:*

(1) $\mathsf{transport}_B(\mathsf{refl}_a) = \mathrm{id}_{B(a)}$ *(identity).*
(2) $\mathsf{transport}_B(\mathsf{trans}(p, q)) = \mathsf{transport}_B(q) \circ \mathsf{transport}_B(p)$ *(composition).*
(3) $\mathsf{transport}_B(\mathsf{symm}(p)) = \mathsf{transport}_B(p)^{-1}$ *(inversion).*

Corollary 9.5 (Dependent application)**.** *Given a dependent function* $f : \prod_{x:A} B(x)$ *and a path* $p : \mathsf{Path}(a, a')$*, there is a dependent path* $\mathsf{apd}(f, p) : \mathsf{Path}^B_p(f(a), f(a'))$ *over* p*.*

9.3. **Failure of UIP.**

Theorem 9.6 (Non-UIP)**.** *The* Path *type does not satisfy the Uniqueness of Identity Proofs principle.*

Proof. Consider any non-empty type A with an element $a : A$. The path refl_a has an empty step list. Construct a *step loop*: a path $\ell : \mathsf{Path}(a, a)$ whose step list contains a non-trivial step (e.g., a β-step followed by its reverse). Since the step lists differ (empty vs. non-empty), $\mathsf{refl}_a \neq \ell$ as elements of $\mathsf{Path}(a, a)$.

More generally, a *step-counting invariant* provides a simple numerical measure separating paths with different rewrite histories. By applying n copies of the step loop, one obtains an infinite family of pairwise distinct loops $\ell^n : \mathsf{Path}(a, a)$ for each $n \in \mathbb{N}$, demonstrating that the loop space is genuinely infinite. □

Remark 9.7. The failure of UIP for Path is not a deficiency but a structural necessity: it is precisely what allows the higher-dimensional structure to exist. If all paths between the same endpoints were equal, the entire tower of 2-cells, 3-cells, etc. would collapse. This should be contrasted with the ambient propositional equality Eq, which *does* satisfy UIP—the two equality notions coexist without contradiction.

10. Applications

10.1. **Combinatorial spaces and fundamental groups.** A *combinatorial space* is presented by generators and relations: points (0-cells), path generators (1-cells), and relations (2-cells). This is the combinatorial analogue of CW-complexes [36] or higher inductive types [63].

Definition 10.1 (Combinatorial space presentation)**.** A *combinatorial space presentation* consists of:

(1) A set P_0 of *points*.
(2) A set P_1 of *path generators*, each with source and target.
(3) A set P_2 of *2-cell relations*, identifying parallel paths.

The fundamental group is computed by reducing loop expressions modulo the groupoid identities G1–G8 and the imposed relations.

Theorem 10.2 (Fundamental group computations)**.** *Using computational paths over combinatorial spaces:*

(a) $\pi_1(S^1) \cong \mathbb{Z}$.
(b) $\pi_1(T^2) \cong \mathbb{Z} \times \mathbb{Z}$.
(c) $\pi_1(\mathbb{R}P^2) \cong \mathbb{Z}/2\mathbb{Z}$.
(d) $\pi_1(K) \cong \mathbb{Z} \rtimes \mathbb{Z}$ *(Klein bottle).*
(e) $\pi_1(\bigvee_n S^1) \cong F_n$ *(bouquet of* n *circles gives the free group on* n *generators).*

Proof of (a). The circle S^1 is presented with one vertex $\star$ and one loop generator $\ell : \mathsf{Path}(\star, \star)$, with no relations imposed. Every loop at $\star$ is a product of copies of ℓ and ℓ^{-1}.

Define the *winding number* $w : \Omega(S^1, \star) \to \mathbb{Z}$ by $w(\ell) = 1$, $w(\ell^{-1}) = -1$, extended multiplicatively: $w(\mathsf{trans}(p, q)) = w(p) + w(q)$. This is well-defined on equivalence classes because the groupoid laws preserve winding number: $w(\mathsf{trans}(\ell, \mathsf{symm}(\ell))) = 1 + (-1) = 0 = w(\mathsf{refl}_\star)$.

Surjectivity: For each $n \in \mathbb{Z}$, the loop ℓ^n (where $\ell^{-k} \coloneqq (\ell^{-1})^k$) has winding number n.

Injectivity: Two loops with the same winding number are rewrite-equivalent. The key step is that any loop expression reduces to a normal form of the shape ℓ^n by cancelling adjacent $\ell \cdot \ell^{-1}$ and $\ell^{-1} \cdot \ell$ pairs (via R5 and R6) and removing units (via R3 and R4). Two expressions with the same winding number reduce to the same ℓ^n.

This establishes the isomorphism $\pi_1(S^1) \cong \mathbb{Z}$. □

Proof of (b). The torus T^2 has one vertex $\star$, two loop generators ℓ_1, ℓ_2, and one relation $\ell_1 \cdot \ell_2 = \ell_2 \cdot \ell_1$. Every loop expression reduces to a normal form $\ell_1^m \cdot \ell_2^n$ for unique $m, n \in \mathbb{Z}$: the commutativity relation allows all copies of ℓ_1 to be moved left of all copies of ℓ_2, and the groupoid laws handle cancellation. The map $[\alpha] \mapsto (w_1(\alpha), w_2(\alpha))$ (winding numbers with respect to each generator) gives the isomorphism $\pi_1(T^2) \cong \mathbb{Z} \times \mathbb{Z}$. □

Proof of (c). The projective plane $\mathbb{R}P^2$ has one vertex $\star$, one loop ℓ, and one relation $\ell^2 = \mathsf{refl}_\star$. Every loop reduces to either $\mathsf{refl}_\star$ (winding number even) or ℓ (winding number odd), giving $\pi_1(\mathbb{R}P^2) \cong \mathbb{Z}/2\mathbb{Z}$. □

Proof of (d). The Klein bottle K has one vertex $\star$, two loop generators ℓ_1, ℓ_2, and the relation $\ell_1 \cdot \ell_2 = \ell_2 \cdot \ell_1^{-1}$. This relation prevents ℓ_1 and ℓ_2 from commuting: instead, ℓ_1 "twists" ℓ_2 by inverting it. The map $[\alpha] \mapsto (w_1(\alpha), w_2(\alpha))$ sends loops to $\mathbb{Z} \times \mathbb{Z}$, but the group operation is not componentwise addition. Instead, $\ell_1 \cdot \ell_2^n = \ell_2^{-n} \cdot \ell_1$ (by the relation), giving the semidirect product structure $\pi_1(K) \cong \mathbb{Z} \rtimes \mathbb{Z}$ where the action is $n \cdot m = (-1)^n m$. □

Example 10.3 (The figure-eight and free groups)**.** The figure-eight (bouquet of two circles) has one vertex and two loop generators a, b with no relations. Its fundamental group is the free group $F_2 = \langle a, b \rangle$. The normal forms are reduced words in $\{a, a^{-1}, b, b^{-1}\}$—exactly the reduced words of the free group, computed by the Dehn algorithm implemented by rules R1–R8. More generally, a bouquet of n circles has fundamental group F_n, the free group on n generators.

Example 10.4 (Lens spaces)**.** A lens space $L(p, 1)$ (with $p \geq 2$) has one vertex $\star$, one loop generator ℓ, and the relation $\ell^p = \mathsf{refl}_\star$. Every loop reduces to ℓ^k for $0 \leq k < p$, giving $\pi_1(L(p, 1)) \cong \mathbb{Z}/p\mathbb{Z}$. The general lens space $L(p, q)$ requires a more delicate analysis involving the interaction of the fundamental domain with the identification.

10.2. **The Seifert–van Kampen theorem.**

Theorem 10.5 (Seifert–van Kampen)**.** *The fundamental groupoid of a pushout is the amalgamated free product of the constituent groupoids. More precisely, if a combinatorial space is presented as a union of two subspaces along a common intersection, then* Π_1 *of the whole is computed from* Π_1 *of the parts.*

This theorem is treated via computational paths in [57], using the groupoid methods of Brown [5, 8].

10.3. **Connections to homotopy type theory.** The relationship between computational paths and HoTT [63] deserves careful delineation. Both frameworks assign non-trivial structure to identity proofs and both arrive at weak ω-groupoid structures. The differences are methodological:

- *Foundation.* HoTT is founded on an axiomatic identity type with path induction (J), augmented by univalence. Computational paths are founded on an explicit term rewriting system.
- *Ambient equality.* Computational paths work over an ambient equality satisfying UIP, while HoTT explicitly rejects UIP. The

two levels coexist without contradiction: Path carries proof-relevant structure in Type, while Eq provides proof-irrelevant equality in Prop.

- *Constructivity.* The univalence axiom in HoTT is not computationally effective in standard Martin-Löf type theory (though cubical type theory [10] provides a computational interpretation). The rewrite rules of computational paths are directly executable.
- *What computational paths add.* Within the rewrite-theoretic setting, computational paths provide: (1) explicit step traces—every equality witness carries a verifiable record of how it was derived; (2) decidable rewrite equivalence via normalisation; (3) a concrete ω-groupoid structure where every coherence witness is a specific derivation, not an abstract existence claim.
- *What HoTT adds.* HoTT provides: (1) the univalence axiom, which identifies type equivalences with paths in the universe—a principle not available in the computational-paths framework; (2) higher inductive types with general path constructors; (3) the ability to work in a "natively homotopical" setting where equality is fundamentally proof-relevant at all levels simultaneously.

A translation dictionary maps the core concepts:

HoTT		**Computational Paths**
Identity type $\mathsf{Id}_A(a,b)$	$\longleftrightarrow$	$\mathsf{Path}(a,b)$
refl_a	$\longleftrightarrow$	refl_a (empty trace)
Path concatenation $p \cdot q$	$\longleftrightarrow$	$\mathsf{trans}(p,q)$
$\mathsf{ap}(f,p)$	$\longleftrightarrow$	$\mathsf{congrArg}(f,p)$
Transport	$\longleftrightarrow$	$\mathsf{transport}(p,x)$
J-eliminator	$\longleftrightarrow$	Derived from contractibility
n-type	$\longleftrightarrow$	Stabilisation dimension
Univalence	$\longleftrightarrow$	Open problem

Remark 10.6 (Two levels of identity). The reader may wonder how UIP can hold in our framework when Theorem 9.6 demonstrated its *failure*. The resolution: two different equality notions are in play. The *ambient equality* $a = b$ is proof-irrelevant and satisfies UIP. The *computational path type* $\mathsf{Path}(a,b)$ is proof-relevant: distinct rewrite traces can determine genuinely distinct path values. These claims are compatible: UIP holds for the ambient equality between path *values*, but distinct values can exist in the first place because the step traces differ.

10.4. **Relationship to cubical type theory.** Cubical type theory [10, 1] provides a computational interpretation of univalence via an interval object $\mathbb{I}$. Paths in cubical type theory are functions $p : \mathbb{I} \to A$ with $p(0) = a$ and $p(1) = b$—literally "continuous maps from an interval."

Computational paths use a fundamentally different mechanism: explicit step traces recording the rewrite history. Where cubical paths are interval-indexed *functions*, computational paths are *sequences of named rewrites*. This gives them a more combinatorial character, well-suited to automated reasoning and decision procedures.

The relationship between these approaches is an active area of investigation. Both achieve a form of "computational content for equality," but via different routes—cubical paths through geometric intuition, computational paths through algebraic rewriting.

10.5. **Directed type theory and higher categories.** The current framework is inherently undirected: every path has an inverse. A *directed* variant, where paths model irreversible computation steps, would connect to directed type theory [58] and $(\infty, 1)$-categories. The key challenge is that the groupoid structure (rules R5–R6) would be replaced by a mere category structure, and the coherence theory would need to account for non-invertible cells.

10.6. **Categorical perspective.** The constructions developed here participate in established categorical frameworks:

(1) The path groupoid $\Pi_1(A)$ is the *free groupoid on a computad*: it is generated by atomic edges (rewrite steps) and relations (the 77 rules).
(2) The hom-objects carry groupoid structure, making the path structure a *2-groupoid*—a category enriched over groupoids.
(3) The double-source structure of 2-cells admits a *double groupoid* interpretation [7].
(4) The rewrite theory connects to Squier's theorem [59] and the higher-dimensional rewriting programme of Guiraud–Malbos [35]: the 77-rule system provides a *coherent presentation* of the path algebra, with critical pairs generating the 2-cells and their resolutions generating 3-cells.

Remark 10.7 (Meaning as use and the groupoid structure). The groupoid structure has a natural reading in terms of the "meaning as use" principle [18, 20, 21]. The rewrite rules specify the *consequences* of equality assertions: what can be derived from knowing that $a = b$ (transitivity with other paths), what follows from the absence of information (refl as the zero-step witness), and how equality interacts with functions

(congruence). The 77-rule system is a complete specification of the "grammar of equality."

This connects to Martin-Löf's observation [44] that equality is "the most important concept in all of mathematics." The computational-paths framework makes explicit what it means to *use* an equality: the groupoid operations are the primitive acts of use, and the rewrite rules specify when two different uses amount to the same thing.

Remark 10.8 (Comparison with related rewriting systems). The 77-rule system occupies a distinctive position in the landscape of term rewriting. A standard presentation of the free groupoid uses essentially the eight rules of Category A. The additional 69 rules express the functoriality and compatibility conditions arising when a groupoid is embedded in a type-theoretic setting.

The work of Squier [59] and its extensions by Guiraud and Malbos [35] connect rewriting systems to higher-dimensional algebra: a confluent and terminating rewrite system gives rise to a coherent presentation. Our system provides such a coherent presentation for the path algebra. In the HoTT setting, the path algebra is *presented* by rules analogous to Category A, but witnessed by identity proofs rather than rewrite steps—our system provides a computational counterpart.

The Knuth–Bendix completion procedure [39] could in principle be used to derive the rule set, but the type-theoretic structure provides a more principled organisation. The key advantage of starting from the type-theoretic principles is that the categories B–G arise systematically from the structure of dependent type theory, rather than being discovered by ad hoc completion.

11. The Lean 4 Formalisation

The central results of this paper have been formalised in the Lean 4 proof assistant [12]. The formalisation comprises over 10,000 theorems and lemmas, all machine-checked, with no uses of `sorry`, `admit`, or custom `axiom` declarations.

11.1. **Architecture.** The formalisation mirrors the mathematical development:

Module	Content
`Path/Basic/`	Path type, step structure, fundamental operations
`Path/Rewrite/Step.lean`	The 77 primitive rewrite steps
`Path/Rewrite/Rw.lean`	Multi-step reduction
`Path/Rewrite/RwEq.lean`	Symmetric rewrite equivalence
`Path/Rewrite/Quot.lean`	Quotienting by rewrite equivalence
`Path/Rewrite/Confluence*.lean`	Church–Rosser confluence
`Path/GroupoidDerived.lean`	Groupoid laws and quotient
`Path/OmegaGroupoid/`	2-cells, 3-cells, ω-groupoid
`Pentagon*.lean`	Pentagon and triangle coherence
`EckmannHilton*.lean`	Eckmann–Hilton argument
`Path/Algebra/`	Deformation theory, homotopy groups
`Path/Applications/`	$\pi_1(S^1)$, Seifert–van Kampen

11.2. **Key design decisions.** *Type vs. Prop.* A crucial design decision is that the step relation and rewrite equivalence target Type rather than Prop:

$$\mathsf{Step} : \mathsf{Path}(a,b) \to \mathsf{Path}(a,b) \to \mathsf{Type}, \qquad \mathsf{RwEq} : \mathsf{Path}(a,b) \to \mathsf{Path}(a,b) \to \mathsf{Type}.$$

If RwEq lived in Prop, all rewrite derivations connecting two paths would be identified by proof irrelevance, collapsing the higher structure. When a Prop-valued wrapper is needed, we use $\mathsf{RwEqProp}(p,q) := \mathsf{Nonempty}(\mathsf{RwEq}(p,q))$.

Inductive 2-cells and 3-cells. The $\mathsf{Derivation}_2$ type is defined as an inductive type in $\mathsf{Type}\,(u+1)$, with constructors for identity, step, inverse, and vertical composition. The $\mathsf{Derivation}_3$ type similarly uses `MetaStep3` constructors that encode the named coherence laws (pentagon, triangle, interchange, inverse/double-inverse coherence, contravariance). In the verification suite, every one of the fourteen named coherence proofs fails if its witness is replaced by $\mathsf{Derivation}_3.\mathsf{refl}$—confirming that the 3-cell layer is genuinely non-trivial.

In Lean 4, the core definitions look as follows. The path type:

```
structure Path (a b :  A) where
steps :  List (Step A)
proof :  a = b
```

The primitive rewrite steps (77 constructors):

```
inductive Step :  Path a b → Path a b → Type where
| symm_refl :  Step (symm (refl a)) (refl a)
| symm_symm :  Step (symm (symm p)) p
| trans_refl_left :  Step (trans (refl a) p) p
| ...
```

The 2-cell type:

```
inductive Derivation2 : Path a b → Path a b → Type
where
| refl (p : Path a b) : Derivation2 p p
| step : Step p q → Derivation2 p q
| inv : Derivation2 p q → Derivation2 q p
| vcomp : Derivation2 p q → Derivation2 q r → Derivation2
p r
```

Whiskering and horizontal composition. Whiskering operations are defined by recursion on the 2-cell constructors, using the congruence rules for trans. Horizontal composition is then defined as in Theorem 7.9.

Verification methodology. The formalisation uses no axioms beyond the standard Lean 4 kernel (including propositional extensionality and quotient types, both of which are axioms in Lean 4 but standard in classical mathematics). No `sorry`, `admit`, or custom `axiom` declarations are used. The build produces over 10,000 lemmas and theorems across approximately 50,000 lines of Lean 4 code.

11.3. **Scale and verification.** The formalisation is publicly available at:

https://github.com/Arthur742Ramos/ComputationalPathsLean

It covers: the groupoid structure; the 77-rule rewrite system (Category A fully formalised); confluence for the completed groupoid fragment; the pentagon and triangle coherences; the Eckmann–Hilton argument; the ω-groupoid structure with contractibility above dimension 3; and the computation $\pi_1(S^1) \cong \mathbb{Z}$ via an encode–decode argument.

A companion formalisation [55] provides a modular Lean 4 framework for confluence and strong normalisation of lambda calculi with products and sums, establishing the metatheoretic foundations on which the path rewrite system rests.

12. Perspectives and Open Problems

Open Problem 12.1 (Extension to dependent types and universes). The framework operates on simple types. Extending to full dependent type theory requires defining *dependent computational paths* $\mathsf{DPath}_B(p)(b, b')$, with step preservation, groupoid law extension, and J-elimination. Universes would require paths between types $\mathsf{Path}(A, B)$.

Open Problem 12.2 (Univalence). Whether computational paths can model the univalence axiom remains open. A partial univalence principle (type equivalences give rise to paths, without the converse) has been established, but full univalence would require a significant extension.

Open Problem 12.3 (Directed computational paths). A directed variant, where paths model irreversible computation steps, would connect to directed type theory [58] and $(\infty, 1)$-categories.

Open Problem 12.4 (Computational complexity). While the 77-rule system terminates and is confluent, the precise computational complexity of normalisation has not been characterised. The normalisation function $\mathsf{normalize}(p) = \mathsf{sc}(\mathsf{toEq}(p))$ computes in time $O(|p|)$ via semantic projection, but normalisation by repeated rule application may require $O(|p|^2)$ steps due to associativity rearrangements.

Open Problem 12.5 (Applications to program verification). Computational paths provide explicit witnesses for program equivalence. Developing this into a practical tool for verified compilation—where compiler transformations are certified by paths between source and target programs—is a natural application.

Open Problem 12.6 (Higher inductive types). A systematic treatment of higher inductive types [63] within the computational-paths framework—with explicit rewrite rules for path constructors—would extend the topological applications significantly. The circle S^1 used in Theorem 10.2 is a simple instance; the general theory would require new rule categories for path constructors and their elimination principles.

Open Problem 12.7 (Grothendieck's homotopy hypothesis). Grothendieck's homotopy hypothesis [34] posits that ω-groupoids model homotopy types. Our weak ω-groupoid construction provides a concrete syntactic candidate. Establishing a formal comparison between the fundamental ω-groupoid $\Pi_\omega(A)$ (defined as the colimit of the computational path tower) and the classifying space $B(\Pi_\omega(A))$ would connect the theory directly to classical homotopy theory.

Open Problem 12.8 (Double groupoid and cubical structure). The 2-cells of computational paths admit two natural source-target structures (vertical and horizontal), giving rise to a *double groupoid* in the sense of Brown–Spencer [7]. Relating this double groupoid to the cubical structure of cubical type theory [10, 6] is an open question.

Open Problem 12.9 (Automation and tooling). The decidability of path equivalence (Theorem 5.9) invites implementation as a tactic in proof assistants. A tactic that automatically normalises path expressions

and checks equivalence would significantly streamline proofs involving equality reasoning. The challenge is efficiency: while the semantic projection is $O(|p|)$, integrating it with the proof assistant's elaboration pipeline requires careful engineering.

13. Conclusion

The theory of computational paths provides a proof-relevant, constructive account of equality in which the *reasons* for equality are first-class mathematical objects. Starting from the labelled natural deduction framework of de Queiroz and Gabbay [32, 31], we have developed a complete rewrite calculus (77 rules, confluent and terminating), shown that it gives rise to a weak ω-groupoid structure (with stabilisation above dimension 3), derived the J-eliminator from contractibility of the based path space, established the failure of UIP, and computed fundamental groups of combinatorial spaces.

The distinctive feature of this approach is its concreteness: every coherence witness is a specific derivation from the rewrite rules, not an abstract existence claim. The guiding principle—that all higher structure is extracted from rewriting—connects proof theory, term rewriting, higher category theory, and algebraic topology in a unified framework. The Lean 4 formalisation (over 10,000 theorems, no axioms) provides machine-checked verification of the central results.

By making the computational content of equality explicit, the theory offers a foundation for studying the higher-dimensional structure of type-theoretic equality that is complementary to—and interacts fruitfully with—the axiomatic approach of homotopy type theory.

Acknowledgements. The authors thank Dov M. Gabbay for his foundational contributions to the programme of labelled deductive systems, and Tiago M. L. de Veras for his contributions to the development of the theory. The first author acknowledges support from Microsoft during the preparation of this work. The second and third authors acknowledge support from CNPq and CAPES (Brazil).

Historical note. The development surveyed here spans over three decades, from de Queiroz's original introduction of labelled equality judgements [15] through the systematic elaboration by de Queiroz, de Oliveira, and Gabbay [32, 22, 13, 14, 24], the connection to identity types by de Queiroz, de Oliveira, and Ramos [25, 52], the groupoid and ω-groupoid constructions [51, 26, 56], and the Lean 4 formalisation [54, 55]. The Seifert–van Kampen theorem for computational

paths was established in [57]. The thesis [49] provided the first comprehensive treatment. The present survey draws on all of these sources.

Appendix A. Complete List of Rewrite Rules

We list all 77 named rewrite clauses in tabular form. Each rule has the form LHS $\to$ RHS; see §4 for detailed discussion.

Category A: Groupoid Core (Rules 1–8).

#	Name	Rule	Description
1	Symm-Refl	$\mathsf{symm}(\mathsf{refl}_a) \to \mathsf{refl}_a$	Inv. of identity
2	Symm-Invol	$\mathsf{symm}(\mathsf{symm}(p)) \to p$	Double inverse
3	Trans-L-Id	$\mathsf{trans}(\mathsf{refl}, p) \to p$	Left identity
4	Trans-R-Id	$\mathsf{trans}(p, \mathsf{refl}) \to p$	Right identity
5	Trans-R-Inv	$\mathsf{trans}(p, \mathsf{symm}(p)) \to \mathsf{refl}$	Right inverse
6	Trans-L-Inv	$\mathsf{trans}(\mathsf{symm}(p), p) \to \mathsf{refl}$	Left inverse
7	Symm-Trans	$\mathsf{symm}(\mathsf{trans}(p, q)) \to \mathsf{trans}(\mathsf{symm}(q), \mathsf{symm}(p))$	Contravariance
8	Trans-Assoc	$\mathsf{trans}(\mathsf{trans}(p, q), r) \to \mathsf{trans}(p, \mathsf{trans}(q, r))$	Associativity

Category B: Type Constructors (Rules 9–25).

#	Name	Description
9	MAP$_2$-DECOMP	Binary congruence → composition of partial maps
10	PROD-β_1	First projection of product path
11	PROD-β_2	Second projection of product path
12	PROD-REC	Product recursor on product path
13	PROD-η	Product path from components = original
14	PROD-SYMM	Inverse distributes to components
15	PROD-FUNC	Componentwise function on product path
16	SIGMA-β_1	First projection of sigma path
17	SIGMA-β_2	Second projection of sigma path
18	SIGMA-η	Sigma path from components = original
19	SIGMA-SYMM	Inverse of sigma path (with transport)
20	SUM-β_1	Sum eliminator on left injection
21	SUM-β_2	Sum eliminator on right injection
22	FUN-β	Function extensionality β-rule
23	FUN-η	Function extensionality η-rule
24	FUN-SYMM	Pointwise inverse of function path
25	APD-REFL	Dependent application on reflexivity

Category C: Transport (Rules 26–32).

#	Name	Description
26	TR-REFL	Transport over reflexivity = identity
27	TR-TRANS	Transport over composition = composite transport
28	TR-SYMM-L	Transport forward then backward = identity
29	TR-SYMM-R	Transport backward then forward = identity
30–32	TR-SIGMA-*	Transport in sigma types (3 rules)

Category D: Context Rules (Rules 33–48).

#	Name	Description
33	Ctx-Congr	Context congruence (meta-rule)
34	Ctx-Symm	Symmetry through contexts
35–36	Ctx-Absorb	Context absorption (left/right)
37	L-Whisker-Intro	$\mathsf{trans}(r, C[p]) \to L_C(r, p)$
38	L-from-R	Left whisker from right whisker
39	L-to-R	Left-to-right whisker conversion
40	R-Whisker-Intro	$\mathsf{trans}(C[p], t) \to R_C(p, t)$
41	R-Assoc	Right whisker associativity
42–45	Whisker-Refl	Whiskers with reflexivity (4 rules)
46–48	Whisker-Idemp	Whisker idempotence (3 rules)

Category E: Dependent Context (Rules 49–60). Twelve rules mirroring Category D for dependent functions, incorporating transport maps. Each non-dependent rule R33–R48 has a dependent counterpart (except R35–R36, R38, R48 which are subsumed by transport coherences).

Category F: Binary Context (Rules 61–68). Eight compatibility rules for binary contexts—left/right, dependent/non-dependent, unary/binary positions.

Category G: Partial Maps (Rules 69–72). Two congruence rules (R69–R70) and two computation rules (R71–R72) for partial map operations arising from binary congruence decomposition.

Category H: Structural Completion (Rules 73–77).

#	Name	Description
73	Symm-Congr	$p \to q \Rightarrow \mathsf{symm}(p) \to \mathsf{symm}(q)$
74	Trans-Congr-L	$p \to q \Rightarrow \mathsf{trans}(p, r) \to \mathsf{trans}(q, r)$
75	Trans-Congr-R	$q \to r \Rightarrow \mathsf{trans}(p, q) \to \mathsf{trans}(p, r)$
76	Cancel-L	$\mathsf{trans}(p, \mathsf{trans}(\mathsf{symm}(p), q)) \to q$
77	Cancel-R	$\mathsf{trans}(\mathsf{symm}(p), \mathsf{trans}(p, q)) \to q$

References

[1] Carlo Angiuli, Guillaume Brunerie, Thierry Coquand, Robert Harper, Kuen-Bang Hou (Favonia), and Daniel R. Licata. Syntax and models of cartesian cubical type theory. *Mathematical Structures in Computer Science*, 31(4):424–468, 2021. doi: 10.1017/S0960129521000347.

[2] Danil Annenkov, Paolo Capriotti, Nicolai Kraus, and Christian Sattler. Two-level type theory and applications. *Mathematical Structures in Computer Science*, 33(8):688–743, 2023. doi: 10.1017/S0960129523000130. arXiv:1705.03307v3.

[3] Michael A. Batanin. Monoidal globular categories as a natural environment for the theory of weak n-categories. *Advances in Mathematics*, 136:39–103, 1998. doi: 10.1006/aima.1998.1724.

[4] Jean Bénabou. Introduction to bicategories. In Jean Bénabou et al., editors, *Reports of the Midwest Category Seminar*, volume 47 of *Lecture Notes in Mathematics*, pages 1–77, Berlin, 1967. Springer-Verlag.

[5] Ronald Brown. *Topology and Groupoids*. BookSurge, 2006. Third edition of Elements of Modern Topology (1968).

[6] Ronald Brown and Philip J. Higgins. On the algebra of cubes. *Journal of Pure and Applied Algebra*, 21:233–260, 1981.

[7] Ronald Brown and Christopher B. Spencer. Double groupoids and crossed modules. *Cahiers de Topologie et Géométrie Différentielle Catégoriques*, 17(4):343–362, 1976.

[8] Ronald Brown, Philip J. Higgins, and Rafael Sivera. *Nonabelian Algebraic Topology: Filtered Spaces, Crossed Complexes, Cubical Homotopy Groupoids*, volume 15 of *EMS Tracts in Mathematics*. European Mathematical Society, 2011.

[9] Alonzo Church and J. Barkley Rosser. Some properties of conversion. *Transactions of the American Mathematical Society*, 39(3):472–482, 1936. doi: 10.2307/1989762.

[10] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical type theory: A constructive interpretation of the univalence axiom. In Tarmo Uustalu, editor, *21st International Conference on Types for Proofs and Programs (TYPES 2015)*, volume 69 of *LIPIcs*, pages 5:1–5:34. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.TYPES.2015.5.

[11] Haskell B. Curry. Functionality in combinatory logic. *Proceedings of the National Academy of Sciences*, 20(11):584–590, 1934. doi: 10.1073/pnas.20.11.584.

[12] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[13] Anjolina G. de Oliveira and Ruy J.G.B. de Queiroz. A normalization procedure for the equational fragment of labelled natural deduction. *Logic Journal of the IGPL*, 7(2):173–215, 1999. doi: 10.1093/jigpal/7.2.173.

[14] Anjolina G. de Oliveira and Ruy J.G.B. de Queiroz. A new basic set of proof transformations. In Sergei Artemov et al., editors, *We Will Show Them: Essays in Honour of Dov Gabbay*, volume 2, pages 499–528. College Publications, London, 2005.

[15] Ruy J.G.B. de Queiroz. A proof-theoretic account of programming and the role of reduction rules. *Dialectica*, 42(4):265–282, 1988. doi: 10.1111/j.1746-8361.1988.tb00919.x.

[16] Ruy J.G.B. de Queiroz. The mathematical language and its semantics: To show the consequences of a proposition is to give its meaning. In Paul Weingartner and Gerhard Schurz, editors, *Reports of the Thirteenth International Wittgenstein Symposium 1988*, volume 18 of *Schriftenreihe der Wittgenstein-Gesellschaft*, pages 259–266. Hölder-Pichler-Tempsky, Vienna, 1989.

[17] Ruy J.G.B. de Queiroz. *Proof Theory and Computer Programming*. PhD thesis, Imperial College, University of London, 1990.

[18] Ruy J.G.B. de Queiroz. Meaning as grammar plus consequences. *Dialectica*, 45(1):83–86, 1991. doi: 10.1111/j.1746-8361.1991.tb00979.x.

[19] Ruy J.G.B. de Queiroz. Grundgesetze alongside Begriffsschrift (abstract). In *Abstracts of the Fifteenth International Wittgenstein Symposium*, pages 15–16, 1992.

[20] Ruy J.G.B. de Queiroz. Meaning, function, purpose, usefulness, consequences – interconnected concepts. *Logic Journal of the IGPL*, 9(5):693–734, 2001. doi: 10.1093/jigpal/9.5.693.

[21] Ruy J.G.B. de Queiroz. On reduction rules, meaning-as-use, and proof-theoretic semantics. *Studia Logica*, 90(2):211–247, 2008. doi: 10.1007/s11225-008-9150-5.

[22] Ruy J.G.B. de Queiroz and Anjolina G. de Oliveira. Term rewriting systems with labelled deductive systems. In *Proceedings of the Brazilian Symposium on Artificial Intelligence (SBIA'94)*, pages 59–72, 1994.

[23] Ruy J.G.B. de Queiroz and Anjolina G. de Oliveira. The functional interpretation of direct computations. *Electronic Notes in*

Theoretical Computer Science, 269:19–40, 2011. doi: 10.1016/j.entcs.2011.03.003.

[24] Ruy J.G.B. de Queiroz, Anjolina G. de Oliveira, and Dov M. Gabbay. *The Functional Interpretation of Logical Deduction*, volume 5 of *Advances in Logic*. World Scientific, Singapore, 2011.

[25] Ruy J.G.B. de Queiroz, Anjolina G. de Oliveira, and Arthur F. Ramos. Propositional equality, identity types, and computational paths. *South American Journal of Logic*, 2(2):245–296, 2016. URL https://www.sa-logic.org/sajl-v2-i2/05-De%20Queiroz-De%20Oliveira-Ramos-SAJL.pdf.

[26] Tiago M.L. de Veras, Arthur F. Ramos, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. Computational paths – a weak groupoid. *Journal of Logic and Computation*, 35(5), 2025. doi: 10.1093/logcom/exad071.

[27] Max Dehn. Über unendliche diskontinuierliche gruppen. *Mathematische Annalen*, 71:116–144, 1911. doi: 10.1007/BF01456932.

[28] Beno Eckmann and Peter J. Hilton. Group-like structures in general categories. I. Multiplications and comultiplications. *Mathematische Annalen*, 145:227–255, 1962. doi: 10.1007/BF01451367.

[29] Gottlob Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens*. Louis Nebert, Halle, 1879. English translation in van Heijenoort (1967).

[30] Gottlob Frege. *Grundgesetze der Arithmetik*, volume I. Hermann Pohle, Jena, 1893.

[31] Dov M. Gabbay. *Labelled Deductive Systems*. Oxford University Press, Oxford, 1996.

[32] Dov M. Gabbay and Ruy J.G.B. de Queiroz. Extending the Curry–Howard interpretation to linear, relevant and other resource logics. *Journal of Symbolic Logic*, 57(4):1319–1365, 1992. doi: 10.2307/2275370.

[33] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50:1–102, 1987.

[34] Alexander Grothendieck. Pursuing stacks (à la poursuite des champs). Manuscript, letter to Daniel Quillen, 1983.

[35] Yves Guiraud and Philippe Malbos. Higher-dimensional categories with finite derivation type. *Theory and Applications of Categories*, 22(18):420–478, 2009.

[36] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, Cambridge, 2002.

[37] Martin Hofmann and Thomas Streicher. The groupoid interpretation of type theory. In Giovanni Sambin and Jan M. Smith,

editors, *Twenty-Five Years of Constructive Type Theory*, pages 83–111. Oxford University Press, 1998.

[38] William A. Howard. The formulae-as-types notion of construction. In Jonathan P. Seldin and J. Roger Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, London, 1980. Original manuscript from 1969.

[39] Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In John Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970.

[40] Joachim Lambek and Philip J. Scott. *Introduction to Higher Order Categorical Logic*, volume 7 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, Cambridge, 1986.

[41] Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 2nd edition, 1998.

[42] Tom Leinster. *Higher Operads, Higher Categories*, volume 298 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, 2004.

[43] Peter LeFanu Lumsdaine. Weak ω-categories from intensional type theory. *Logical Methods in Computer Science*, 6(3):1–19, 2010. doi: 10.2168/LMCS-6(3:24)2010.

[44] Per Martin-Löf. Correctness of assertion and validity of inference. Lecture at the Rolf Schock Symposium, Stockholm, 2022. Slightly edited transcript of lecture delivered on 26 October 2022. https://pml.flu.cas.cz/uploads/PML-Stockholm26Oct22.pdf.

[45] J. Peter May. *A Concise Course in Algebraic Topology*. Chicago Lectures in Mathematics. University of Chicago Press, 1999.

[46] M. H. A. Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942. doi: 10.2307/1968867.

[47] Ulf Norell et al. Agda, 2024. URL https://wiki.portal.chalmers.se/agda/pmwiki.php.

[48] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Number 3 in Stockholm Studies in Philosophy. Almqvist & Wiksell, Stockholm, 1965.

[49] Arthur F. Ramos. *Explicit Computational Paths in Type Theory*. PhD thesis, Universidade Federal de Pernambuco, 2019. Abstract published in Bulletin of Symbolic Logic, 25(2):213–214, 2019. doi:10.1017/bsl.2019.2.

[50] Arthur F. Ramos, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. Sequences of rewrites: A categorical interpretation. arXiv:1412.2105, 2014.

[51] Arthur F. Ramos, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. On the groupoid model of computational paths. arXiv:1506.02721, 2015.

[52] Arthur F. Ramos, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. On the identity type as the type of computational paths. *Logic Journal of the IGPL*, 25(4):562–584, 2017. doi: 10.1093/jigpal/jzx015.

[53] Arthur F. Ramos, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. Explicit computational paths. *South American Journal of Logic*, 4(2):441–484, 2018. Preprint arXiv:1609.05079, 2016.

[54] Arthur F. Ramos, Anjolina G. de Oliveira, Ruy J.G.B. de Queiroz, and Tiago M.L. de Veras. Formalizing computational paths and fundamental groups in Lean. arXiv:2511.19142, 2025.

[55] Arthur F. Ramos, Anjolina G. de Oliveira, Ruy J.G.B. de Queiroz, and Tiago M.L. de Veras. A modular Lean 4 framework for confluence and strong normalization of lambda calculi with products and sums. arXiv:2512.09280, 2025.

[56] Arthur F. Ramos, Tiago M.L. de Veras, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. Computational paths form a weak ω-groupoid. arXiv:2512.00657, 2025.

[57] Arthur F. Ramos, Tiago M.L. de Veras, Ruy J.G.B. de Queiroz, and Anjolina G. de Oliveira. The Seifert–van Kampen theorem via computational paths: A formalized approach to computing fundamental groups. arXiv:2512.03175, 2025.

[58] Emily Riehl and Michael Shulman. A type theory for synthetic ∞-categories. *Higher Structures*, 1(1):147–224, 2017.

[59] Craig C. Squier. Word problems and a homological finiteness condition for monoids. *Journal of Pure and Applied Algebra*, 49(1–2): 201–217, 1987. doi: 10.1016/0022-4049(87)90129-0.

[60] James Dillon Stasheff. Homotopy associativity of H-spaces. I, II. *Transactions of the American Mathematical Society*, 108:275–312, 1963. doi: 10.2307/1993608.

[61] William W. Tait. Intensional interpretations of functionals of finite type I. *Journal of Symbolic Logic*, 32(2):198–212, 1967. doi: 10.2307/2271658.

[62] The Coq Development Team. The Coq proof assistant, 2024. URL https://coq.inria.fr/.

[63] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced

Study, Princeton, NJ, 2013. URL https://homotopytypetheory.org/book.

[64] Benno van den Berg and Richard Garner. Types are weak ω-groupoids. *Proceedings of the London Mathematical Society*, 102 (2):370–394, 2011. doi: 10.1112/plms/pdq026.

[65] Vladimir Voevodsky. A simple type system with two identity types. Unpublished notes, 2013. URL https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/HTS.pdf.

[66] Vladimir Voevodsky. Homotopy type theory and formalization of mathematics. Lecture slides, Institute for Advanced Study, 2014. Available at https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/2014_IAS.pdf.

Microsoft Corporation
Email address: arfreita@microsoft.com

Centro de Informática, Universidade Federal de Pernambuco (CIn/UFPE)
Email address: ruy@cin.ufpe.br

Centro de Informática, Universidade Federal de Pernambuco (CIn/UFPE)
Email address: ago@cin.ufpe.br